Robotics 2

Artificial Intelligence and Robotics

logo.png

Martina Doku Giuseppina Iannotti Gianmarco Scarano

December 16, 2024

Contents

1 Calibration

Calibration goal: Recover as much as possible pose errors by correcting the nominal **??** set of D-H parameters, based on independent external measurements

1.1 Error measure

The error measure is the difference between the true value r and the measured value r_{nom}. It is given by:

$$\Delta r = r - r_{nom} = \frac{\partial f}{\partial \alpha}\Delta\alpha + \frac{\partial f}{\partial a}\Delta a + \frac{\partial f}{\partial d}\Delta d + \frac{\partial f}{\partial \theta}\Delta\theta \tag{1}$$

where $\frac{\partial f}{\partial \alpha}$, $\frac{\partial f}{\partial a}$, $\frac{\partial f}{\partial d}$, $\frac{\partial f}{\partial \theta}$ are the partial jacobians (f is the forward kinematic) evaluated in nominal conditions

1.2 Calibration Equation

The calibration equation is given by:

$$\Delta\psi = \begin{bmatrix} \Delta\alpha \\ \Delta a \\ \Delta d \\ \Delta\theta \end{bmatrix} \tag{2}$$

$$\phi = \begin{bmatrix} \frac{\partial f}{\partial \alpha} & \frac{\partial f}{\partial a} & \frac{\partial f}{\partial d} & \frac{\partial f}{\partial \theta} \end{bmatrix} \tag{3}$$

$$\Delta r = \phi \cdot \Delta\psi \tag{4}$$

Note: If we have more unknowns than equations, we need to perform l experiments to solve the calibration equation. Thus we obtain:

$$\Delta\bar{r} = \bar{\phi} \cdot \Delta\psi \tag{5}$$

where $\Delta\bar{r}$ is the average error and $\bar{\phi}$ is the regressor matrix evaluated at nominal values, and $\Delta\psi$ is the unknowns.

1.3 Calibration Algorithm

The calibration algorithm is performed using the following steps:

1. $\Delta\bar{r} = \bar{\phi} \cdot \Delta\psi$
2. $\Delta\psi = \bar{\phi}^{\#}\Delta\bar{r} = (\bar{\phi}^T \cdot \bar{\phi})^{-1} \cdot \bar{\phi}^T \cdot \Delta\bar{r}$
3. $\psi_{nom} + \Delta\psi = \psi'$
4. $\Delta\bar{r}' = \bar{\phi}' \cdot \Delta\psi$ where $\bar{\phi}'$ is the regressor matrix evaluated at the new values.
5. repeat from step 2

Note: we use the pseudo-inverse to solve the calibration equation, it returns the minimum norm solution in case of underdetermined system, the minimum error solution in case of overdetermined system.

1.3.1 ex: Write regressor matrix from a DH table

The first step is to obtain the position vector from the DH table. Then, for each component (a, α, d, θ) that we are interested in, we build the jacobian $\frac{\partial f}{\partial \alpha}$... . The regressor equation is then given by:

$$\Delta p = \frac{\partial f}{\partial a}\Delta a + \frac{\partial f}{\partial \alpha}\Delta \alpha + \frac{\partial f}{\partial d}\Delta d + \frac{\partial f}{\partial \theta}\Delta \theta = \Phi \Delta \phi \tag{6}$$

with:

$$\Phi = \begin{bmatrix} \frac{\partial f}{\partial a} & \frac{\partial f}{\partial \alpha} & \frac{\partial f}{\partial d} & \frac{\partial f}{\partial \theta} \end{bmatrix} \tag{7}$$

Note: if we are interested ony in the fine tuning of some parameters, we can exclude the others from all the equations.

1.3.2 ex: Find optimal link length from positions and joints

The steps are the following:

1. Compute the position vector from the forward kinematics on the given joint measures and expected link lengths (if given)
2. Check if the position vector is the same as the one measured, if not go to step 3
3. rewrite the position vector as a function of the link lengths:
   $\Delta p = p - p_{nom} = \phi(q)\Delta l$
   $\Delta p = \begin{bmatrix} \Delta p_a \\ \Delta p_b \\ \Delta p_c \\ \Delta p_d \end{bmatrix} = \begin{bmatrix} \phi(q_a) \\ \phi(q_b) \\ \phi(q_c) \\ \phi(q_d) \end{bmatrix} \Delta l$ **Note:** Δp_a and co. are vertical vectors, they must be stacked vertically (if i have 3 measures in 2 dim, i obtain a 6x1 vector)
4. where a, b, c, d are the different joint measures
5. solve the calibration equation for l as: $\Delta l = \Phi^{\#} \Delta p$

1.3.3 ex: Theoretical question

Question: When is it convenient to use a two steps calibration?
Answer: It is convinient to use a two steps calibration when it is expected that the DH parameters will have very different uncertainty ranges.
The first step is performed only on high uncertainty parameters, the second on all of them. This improves the accuracy of the pseudoinverse of the regressor matrix

2 Inverse Kinematics with Redundancy

Types of redundancy:

- **kinematic redundancy**: the robot has more degrees of freedom than needed
- **components redundancy**: the robot has more components than needed

- **control redundancy**: the robot is redundant in its supervision architecture

The resolution of redundancy can be done using two methods:

1. **Local methods**: the local methods are online methods that are used to solve the redundancy problem. They are Linear Quadratic problems. (discrete approach)
2. **Global methods**: the global methods are offline methods. They are used to solve the redundancy problem. They are a non-linear problem.

2.1 Which method to choose?

If the exercise asks for the solution that:

- **minimizes the kinetic energy**: use the weighted pseudo-inverse with the inertia matrix as weight
- **minimizes the torque norm (or task error norn)**: use the pseudo-inverse
- **minimize norm of joint velocities**: use the pseudo-inverse
- **minimizes the error on the higher priority task**: use the task priority (recursive) method
- **minimizes the error on the lower priority task while correctly executing the higher priority one**: use the nullspace method or task priority (recursive) method (the second is better if you have more than 2 tasks)
- **minimizes the acceleration norm**: use the nullspace method as in **??**

Note: if asked to find another solution without specific requirements, it is usually a solution that you can infer by looking at the jacobian and the desired velocity, without the need of any specific method.

2.2 Local methods

The three kinds of local methods are:

1. **Jacobian-based methods**: among the infinite solutions, one is chosen, e.g., that minimizes a suitable (possibly weighted) norm. The problem of this method is that it might encounter singularities and it can lead to non repeatable position in joint space.
2. **Null-space methds**: a term is added to the previous solution so as not to affect execution of the task trajectory
3. **Task-augmentation methods**: redundancy is reduced/eliminated by adding $S \leq N - M$ additional tasks

2.3 Jacobian-based methods

The Jacobian-based methods are used to solve the redundancy problem. They give a solution in the form:

$$\dot{q} = K(q) \cdot \dot{r} \tag{8}$$

where K is a generalized inverse of the Jacobian matrix. Possible choices for K are:

- Pseudo-inverse **??**
- Weighted pseudo-inverse **??**
- Damped Least Square pseudo-inverse **??**: this method is used to avoid singularities.

2.3.1 Pseudo-inverse

If J is full rank, the pseudo-inverse is given by:

$$J^{\#} = J^T (J^T J)^{-1} \tag{9}$$

If J is not full rank, the pseudo-inverse it is computed by using the SVD decomposition (or `pinv` in Matlab).
Note: the pseudo-inverse is not unit independent.

2.3.2 Weighted pseudo-inverse

The weighted pseudo-inverse is given by:

$$J^{\#} = W^{-1} J^T (J W^{-1} J^T)^{-1} \tag{10}$$

where larger weights are given to the most important tasks, usally the weights are proportional to the inverse of the joint ranges.
Note: It is NOT a pseudoinverse but it shares some properties **??**.
Note: To find the velocity that **minimizes the Kinetic energy**, we have to use the weighted pseudo-inverse with the **Inertia matrix** as weight.

2.3.3 Damped Least Square pseudo-inverse

The damped least square pseudo-inverse is given by:

$$J^{\#} = J^T (J J^T + \mu^2 I)^{-1} \tag{11}$$

where μ is a damping factor (the larger μ, the larger the error)
It is a compromise between large joint velocity and task accuracy, it's used to obtain **robust behavior in the presence of singularities**, but in its basic version it always gives a task error.

2.4 Null-space methods

The null space methods are used to solve the redundancy problem. They give a solution in the form:

$$\dot{q} = J^{\#} \cdot \dot{r} + (I - J^{\#}J) \cdot \dot{q}_0 \tag{12}$$

where $\dot{q}_0$ is a generic preferred joint velocity.
The equation is the sum of the solution of the primary task and the orthogonal projection of $\dot{q}_0$ on the null space of J (The null space $\mathcal{N}$ represents the space of joint velocities that do not affect the end-effector position) We have mainly two methods:

- **Projected gradient method** $\Rightarrow$ **??**
- **Reduced gradient method** $\Rightarrow$ **??**

2.4.1 Projected gradient method

The solution for the projected gradient method is obtained by the minimization of the following function:

$$H = \frac{1}{2}(\dot{q} - \dot{q}_0)^T W (\dot{q} - \dot{q}_0) \tag{13}$$

The solution is given by:

$$\dot{q} = J^{\#} \cdot \dot{r} + (I - J^{\#}J) \cdot \dot{q}_0 \tag{14}$$

or

$$\dot{q} = J^{\#} \cdot \dot{r} + (I - J^{\#}J) \cdot \nabla H(q) \tag{15}$$

Note: there is a necessary condition of constrained optimality: $(I - J^{\#}J)\nabla H(q) = 0$
Possible values of H
The possible values of H are:

- manipulanility: $H = \sqrt{det(JJ^T)}$
- joint limits: $H_{range} = \frac{1}{2N}\sum_{i=1}^{n} \frac{(q_i - \bar{q}_i)^2}{(q_{max_i} - q_{min_i})^2}$ **Note:** In this case $\dot{q}_0 = -\nabla H_{range}$, we introduce the minus because we want to `minimize` the distance from the center of the joint range.
- obstacle avoidance: $H = \min\limits_{a \in robot} \min\limits_{b \in obstacle} \|a(q) - b\|^2$

2.4.2 Reduced gradient method

The solution is given by:

$$\dot{q} = \begin{bmatrix} \dot{q}_a \\ \dot{q}_b \end{bmatrix} = \begin{bmatrix} J_a^{-1} \\ 0 \end{bmatrix} \dot{r} + \begin{bmatrix} -J_a^{-1}J_b \\ I \end{bmatrix} \cdot \begin{bmatrix} -(J_a^{-1}J_b)^T & I \end{bmatrix} \nabla_q H \tag{16}$$

Where J_a is the greatest non singular minor of J and J_b is the other minor.
Finding the non singular minor
The non singular minor can be found by using the following steps:

1. Compute the Jacobian J

2. Compute the rank r of J
3. Compute the determinant of all submatrices of J of size r
4. Select the minor with the greatest (absolute) determinant (it must be non singular, so the determinant must be different from 0)

2.4.3 Resolution at acceleration level

The resolution at acceleration level is given by:

$$\ddot{q} = J(q)^{\#}(\ddot{r} - \dot{J}(q)\dot{q}) + (I - J(q)^{\#}J(q))\ddot{q}_0 \tag{17}$$

where $\ddot{q}_0$ is a generic preferred joint acceleration, that can be written as: $\ddot{q}_0 = -\nabla H_{range} - K_D\dot{q}$ where K_D is a damping factor. This yields the minimum acceleration norm solution.

2.4.4 ex: Find the joint velocity that minimizes the H_{range} and scale it if not feasible

The steps are the following:

1. Use the projected gradient method to find the joint velocity that minimizes the H_{range}
2. If the joint velocity is not feasible we want to find a scaling factor k that makes it feasible. We want $\dot{r}new = k\dot{r}$
3. Since we can write the new version of the $\dot{q}$ (that is $\dot{q}_{max}$) by substituting $\dot{r}$ with $\dot{r}new$ in the projected gradient method equation
4. We extract k from the equation, and we compute the new $\dot{r}$ as $\dot{r}new = k\dot{r}$

2.4.5 ex: Find a solution in case of presence of task error

The steps are the following:

1. To find the acceleration that brings the task error back to zero we modify the tsandard equation by adding the PD terms: $\ddot{q} = -K_v\dot{q} + J(q)^{\#}(\ddot{r} + K_D(\dot{r}_d - J_r(q)\dot{q}) + K_P(r_d - p_x(q))) - (I - J(q)^{\#}J(q))\ddot{q}_0 + J_r(q)K_v\dot{q}$

2.5 Task augmentation methods

The task augmentation methods are used to solve the redundancy problem.

2.5.1 Task Priority

This method gives a solution that satisfies the several task in priority order in the following way:

$$\dot{q} = J_1^{\#}\dot{r}_1 + (I - J_1^{\#}J_1)v_1 \tag{18}$$

where v_1 is made to satisfy also (possibly) the lower priority task.

$$v_1 = (J_2P_1)^{\#}(\dot{r}_2 - J_2J_1^{\#}\dot{r}_1) + (I - (J_2P_1)^{\#}(J_2P_1))^{\#}v_2 \tag{19}$$

2.5.2 General recursive task priority formulation

In the general recursive task priority formulation, we have to do the following:

1. We start with $\dot{q}_0 = 0$ and $P_{A0} = I$
2. We compute the solution at k-th level as: $\dot{q}_k = \dot{q}_{k-1} + (J_k P_{A,k-1})^{\#}(\dot{r}_k - J_k \dot{q}_{k-1})$
3. We compute the projection matrix as: $P_{A,k} = P_{A,k-1} - (J_k P_{A,k})^{\#} J_k P_{A,k-1}$

where $P_{A,k-1} = I - J^{\#}_{A,k-1} J_{A,k-1}$ and $J_{A,k-1}$ is the jacobian of the $k-1$ task.

2.5.3 ex: Check if the tasks can be executed in parallel

The steps are the following:

1. Compute the Jacobian of the two tasks
2. Compute the rank of the two Jacobians and the rank of the extended Jacobian
3. If the rank of the extended Jacobian is equal to the sum of the ranks of the two Jacobians, the tasks can be executed in parallel
4. If the rank of the extended Jacobian is less than the sum of the ranks of the two Jacobians, the tasks can not be executed in parallel

2.6 SNS method

The SNS method is used to solve the redundancy problem with hard constraints. It saturates one overdriven joint command at a time, until a feasible and better performing solution is found.

$$\dot{q} = (JW)^{\#} s\dot{x} + (I - (JW)^{\#} J)\dot{q}_N \tag{20}$$

where s is the scaling factor, W is the diagonal 0-1 matrix , $\dot{q}_N$ contains saturated joint velocities.
The algorithm is the following:

1. Initialize W matrix with 1 on the diagonal if the joint is enabled, 0 if it is not
2. Compute the solution as: $\dot{q} = J^{\#}\dot{r}$
3. Check joint velocity limits: $\dot{q}_{min} \leq \dot{q} \leq \dot{q}_{max}$
4. Compute task scaling factor on the most critical joint
5. If a larger scaling factor is found, update W matrix
6. Disable the most critical joint by forcing it to its max velocity and repeat from step 2

2.6.1 ex: Find the acceleration command that satisfies the limits

We have to do the following:

1. Compute the acceleration command $\ddot{q} = J(q)^{\#}(\ddot{p} - \dot{J}(q)\dot{q})$
2. Check if the acceleration command satisfies the limits
3. $\ddot{Q}_{min} = \max(A_{min}, \frac{V_{min}-\dot{q}}{T_c})$
4. $\ddot{Q}_{max} = \min(A_{max}, \frac{V_{max}-\dot{q}}{T_c})$
5. We have to check that $\ddot{Q}_{min} \leq \ddot{q} \leq \ddot{Q}_{max}$
6. If not we do:
7. $\ddot{p}_{new} = \ddot{p} - J_i A_i$ where i is the index of the most exceeding joint J_i is the i-th column of the Jacobian and A_i is the i-th joint max acceleration
8. Re-compute $\ddot{q}$ as $\ddot{q} = J(q)^{\#}_{new}(\ddot{p}_{new} - \dot{J}(q)_{new}\dot{q}_{new})$ where $J(q)_{new}$ is the Jacobian with the i-th column removed and $J(q)_{new}$ is the i-th joint max acceleration

2.6.2 ex: Find the joint velocity that executes the task and that satisfies the joint velocities bound

We have to do the following:

1. Compute $\dot{q} = J(q)^{\#}\dot{p}$
2. Check if the joint velocity satisfies the limits
3. If not find the most exceeding joint i and recompute the velocity $\dot{p}$ as $\dot{p}_{new} = \dot{p} - J_i * V_i$ where J_i is the i-th column of the Jacobian and V_i is the i-th joint max velocity
4. Re-compute $\dot{q}$ as $\dot{q} = [(J_1 \dots J_{i-1} J_{i+1} \dots J_n)^{\#}\dot{p}_{new}]$

2.6.3 ex: Find the joint velocity with minimum norm that executes the task

The steps are the following:

1. Compute the Jacobian J
2. Compute the rank of J
3. If the rank of J is full, use the pseudo-inverse
4. Otherwise use the pseudoinverse on J_i containing the J with only the independent rows use the corresponding rows in $\dot{r}$

3 Dynamic redundancy resolution

The dynamic redundancy resolution is used to solve the redundancy problem. We have the Linear Quadratic problem:

$$J(q)\ddot{q} = \ddot{x} = \ddot{r} - \dot{J}(q)\dot{q} \tag{21}$$

typical objectives are:

- **torque norm**: $H(q) = \frac{1}{2} * ||\tau||^2$
  soution: $\tau = (J(q)M^{-1}(q))^{\#}(\ddot{r} - \dot{J}(q)\dot{q} + J(q)M^{-1}(q)n(q,\dot{q}))$
  when: good for short trajectories, otherwise it can lead to torque oscillations
- **(squared inverse inertia weighted) torque norm**: $H(q) = \frac{1}{2}||\tau||^2_{M^{-2}}$
  soution: $\tau = M(q)J^{\#}(q)(\ddot{r} - \dot{J}(q)\dot{q} + J(q)M^{-1}(q)n(q,\dot{q}))$
  when: to be preferred, good performance in general
- **(inverse inertia weighted) torque norm**: $H(q) = \frac{1}{2}||\tau||^2_{M^{-1}}$
  soution: $\tau = J^T(q)(J(q)M^{-1}(q)J^T(q))^{-1} * (\ddot{r} - \dot{J}(q)\dot{q} + J(q)M^{-1}(q)n(q,\dot{q}))$

Note: $n(q,\dot{q})$ is the sum of (coriolis, centrifugal, gravity) torques $c(q,\dot{q})$ and $g(\dot{q})$.

3.1 Linear Quadratic problem

To solve a linear quadratic problem we have to follow the following steps:

1. Start from defining the objective function as: $H(q) = \frac{1}{2}(\ddot{q} - \ddot{q}_0)^T W(\ddot{q} - \ddot{q}_0)$ or $H(q) = \frac{1}{2}(\dot{q} - \dot{q}_0)^T W(\dot{q} - \dot{q}_0)$ (identify your weight matrix from there)
2. We have to set the constraints as $Jx = y$ and we have to identify the value of x and y in our case (typically if x is $\ddot{q}$, y is $\ddot{r} - \dot{J}\dot{q}$ while if x is $\dot{q}$, y is $\dot{r}$)
3. Find x_0 by setting the derivative of the objective function to 0
4. We have to compute the solution as: $x = x_0 + W^{-1}J^T(JW^{-1}J^T)^{-1}(y - Jx_0)$

3.1.1 Standard velocity case

The Linear Quadratic problem is given by:

$$\min H(q) = \frac{1}{2}(\dot{q} - \dot{q}_0)^T W(\dot{q} - \dot{q}_0) \tag{22}$$

s.t.

$$J(q)\dot{q} = \dot{r} \tag{23}$$

The solution is given by:

$$\dot{q} = \dot{q}_0 + J^{\#}(q)_W(\dot{r} - J(q)\dot{q}_0) \tag{24}$$

which is equal to:

$$\dot{q} = J^{\#}_W\dot{r} + (I - J^{\#}_W J)\dot{q}_0 \tag{25}$$

3.2 Quadratic programming

The quadratic programming is used to solve the redundancy problem, it is a generalization of the linear quadratic problem. The general form of the quadratic programming is given by:

$$\min \frac{1}{2}||J(q)\ddot{q} - \ddot{r}||^2 + \frac{1}{2}||\omega||^2 \tag{26}$$

s.t.

$$C(q, \dot{q})\dot{q} - \omega \leq d \tag{27}$$

with ω the slack variable, $C(q, \dot{q})$ the constraint matrix and d the constraint vector.
The solution is given by: TBD

4 Dynamic Model

The **dynamic model** of a robot provides the relationship between the generalized forces u and the assumed configurations over time $q(t)$ in form of a set of differential equations $\phi(q, \dot{q}, \ddot{q}) = u$
We have mainly two ways to obtain the dynamic model of a robot:

1. **Euler-Lagrangian method**: To obtain the dynamic model of a robot in **closed form**
   best for : Studying the dynamic properties and control schemes

2. **Newton-Euler method**: To obtain the dynamic model of a robot in a **recursive way**.
   best for : Implementation of control schemes

5 Euler-Lagrangian method

5.1 Euler Lagrange equation

The Euler Lagrange equation is given by:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}}\right) - \frac{\partial L}{\partial q} = u \tag{28}$$

where L is the Lagrangian of the robot and u are the non conservative forces.
The Lagrangian is given by:

$$L(q, \dot{q}) = T(q, \dot{q}) - U(q) \tag{29}$$

where T is the kinetic energy and U is the potential energy.

5.2 Dynamic equation

The full dynamic equation is given by:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = u \tag{30}$$

where:

- $M(q)$ is the inertia matrix
- $C(q, \dot{q})$ is the coriolis and centrifugal matrix
- $G(q)$ are the gravity matrix
- u is the non conservative forces
- $\ddot{q}$ is the joint acceleration
- $\dot{q}$ is the joint velocity
- q is the joint position

Alternatively we can write the dynamic equation as:

$$M(q)\ddot{q} + S(q, \dot{q})\dot{q} + G(q) = u \tag{31}$$

5.2.1 ex: Find the dynamic equation of the robot

The steps are the following:

1. Start from defining the kinetic energy of the robot as: $T_i = \frac{1}{2}m_i\dot{r_i}^2 + \frac{1}{2}\omega_i^T I\omega_i$
2. Compute the kinetic energy of the robot as: $T = \sum_{i=1}^{n} T_i$
3. Compute the inertia matrix as: $M(q) = 2 * \frac{\partial T}{\partial \dot{q}^2}$
4. Find the coriolis and centrifugal matrix as in **??**
5. Find the gravity vector as in **??**
6. Compute the friction torque
7. Compute u as: $u = \tau - \tau_{friction}$ where τ is the commanded torque
8. Compute the dynamic equation as: $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = u$

5.3 Inertia matrix

The inertia matrix is a matrix that contains the inertia of the robot, it can be derived by the following **foundamental relation**:

$$T = \frac{1}{2}\dot{q}^T M(q)\dot{q} \tag{32}$$

where T is the kinetic energy of the robot.

5.3.1 Properties

The properties of the inertia matrix are:

- $M(q)$ is symmetric
- $M(q)$ is positive definite
- $M(q)$ is full rank

Note: The inertia matix is never a function of the first coordinate q1.

5.3.2 Standard body inertia matrix

- **parallelepiped**: given sides a (height), b (width), c (depth) and mass m, the inertia matrix is given by:

$$I = \frac{m}{12}\begin{bmatrix} b^2+c^2 & 0 & 0 \\ 0 & a^2+c^2 & 0 \\ 0 & 0 & a^2+b^2 \end{bmatrix} \tag{33}$$

- **empty cylinder**: given inner radius a and outer radius b, height c and mass m, the inertia matrix is given by:

$$I = \frac{m}{12}\begin{bmatrix} 6(a^2+b^2) & 0 & 0 \\ 0 & 3a^2+3b^2+c^2 & 0 \\ 0 & 0 & 3a^2+3b^2+c^2 \end{bmatrix} \tag{34}$$

fundamental kinematic relation of rigid bodies: $v = v_{cm} + \omega \times r = v_{cm} + S(\omega)r$

5.3.3 ex: Find the inertia matrix of a robot given the DH table and position of CoMs

The steps are the following:

1. The first thing we want to do is to compute the T matrix for each link as in **??**
2. Now we can compute all the T_i and the final T as $T = \sum_{i=1}^{n} T_i$
3. Finally we can obtain the inertia matrix as: $M(q) = 2 * \frac{\partial T}{\partial \dot{q}^2}$

5.3.4 ex: Check if a matrix is an inertia matrix

The steps are the following:

1. Check that the matrix is not dependent from the first coordinate **(note, it can be dependent from the difference between the first and another coordinate, but not directly from the first coordinate)**
2. Check if the matrix is symmetric
3. Check if the matrix is positive definite:

- Check if the determinant is negative, in that case the matrix is not positive definite, if it is positive continue
- Compute the eigenvalues of the matrix
- Check if all the eigenvalues are positive, if any eigenvalue is negative the matrix is not positive definite

5.4 Coriolis and centrifugal matrix

The coriolis and centrifugal matrix is given by:

$$c_i(q, \dot{q}) = \dot{q}^T C_i(q) \dot{q} \tag{35}$$

where $C_i(q)$ is the matrix of **Christoffel symbols** obtained as:

$$C_i(q) = \frac{1}{2}\left(\frac{\partial M_i}{\partial q} + (\frac{\partial M_i}{\partial q})^T - \frac{\partial M}{\partial q_i}\right) \tag{36}$$

where M_i is the i-th column of the inertia matrix.

5.5 Kinetic Energy

The kinetic energy of a link is given by: (Konig theorem)

$$T_i = \frac{1}{2} m_i \cdot^0 v_{c,i}^T \cdot^0 v_{c,i} + \frac{1}{2} \omega_i^T I_{c,i} \omega_i \tag{37}$$

Note: ω_i and $I_{c,i}$ should be expressed in the same reference frame, but the product $\omega_i^T I_{c,i} \omega_i$ is invariant w.r.t. any chosen frame
An alternative way to compute the kinetic energy is:

$$T = \frac{1}{2} \dot{q}^T M(q) \dot{q} \tag{38}$$

5.5.1 Moving frames algorithm

The moving frames algorithm is used to compute the kinetic energy of a robot. The steps are the following:

1. Initialize ${}^0v_0 = 0$ and ${}^0\omega_0 = 0$
2. Define $\sigma = 0$ if the joint is revolute, $\sigma = 1$ if the joint is prismatic
3. Compute ${}^i\omega_i =^{i-1} R_i^T(q_i) \cdot [{}^{i-1}\omega_{i-1} + (1-\sigma)\dot{q}_i^{i-1} z_{i-1}]$
4. Compute
$$ {}^iv_i =^{i-1} R_i^T(q_i) \cdot [{}^{i-1}v_{i-1} + \sigma_i \dot{q}_i^{i-1} z_{i-1} +^{i-1} \omega_i \times^{i-1} r_{i-1,i}] \tag{39}$$
$$ =^{i-1} R_i^T(q_i) \cdot [{}^{i-1}v_{i-1} + \sigma_i \dot{q}_i^{i-1} z_{i-1}] +^i \omega_i \times^i r_{i-1,i} \tag{40}$$
5. Compute ${}^iv_{c,i} =^i v_i +^i \omega_i \times^i r_{c,i}$
6. Compute $T_i = \frac{1}{2} m_i^i v_{c,i}^T v_{c,i} + \frac{1}{2} \cdot^i \omega_i^T I_{c,i}^i \omega_i$

where ${}^ir_{c,i}$ is the distance of the CoM from frame i, ${}^ir_{i-1,i}$ is the distance of the joint i from frame i-1, ${}^{i-1}z_{i-1}$ is the z axis $\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^T$

5.5.2 ex: Find the kinetic energy of a robot

To obtain the kinetic energy of a robot we must calculate the kinetic energy of each link and sum them up.
To obtain T_i we have to do the following:

1. Check if the link is prismatic or revolute
2. If it is prismatic:
   - we just have to compute the linear velocity of the center of mass, that is given by: the linear velocities of the previous link + the linear velocity of the center of mass of the link expressed in function of $\dot{q}_i$
   - we can write the kinetic energy as: $T_i = \frac{1}{2} m_i \cdot^0 v_{c,i}^T \cdot^0 v_{c,i}$
3. If it is revolute:
   - we have to compute the linear velocity ${}^0v_{c,i}$ of the center of mass, that is given by ${}^0\dot{p}_{c,i}$
   - we have to compute the angular velocity ω_i of the link, that is given by the sum of the angular velocities of the previous links and the current one (that is $\dot{q}_i$)
   - we can write the kinetic energy as: $T_i = \frac{1}{2} m_i \cdot^0 v_{c,i}^T \cdot^0 v_{c,i} + \frac{1}{2} \omega_i^T I_{c,i} \omega_i$
4. Finally we can sum all the T_i to obtain the kinetic energy of the robot

5.6 Potential Energy

The potential energy of a link is given by:

$$U_i = -m_i g^T r_{0,ci} \tag{41}$$

where: $r_{c,i}$ is the distance from the center of mass to the reference frame and **g** is the gravity vector (usually $[0, -9.81, 0]^T$).
Note: Generally, you'll find that the professor does not use the negative sign in the formula, that's because if $g = -9.81$, the minus sign gets canceled out and he directly skips this step.
Note: If you are given the distance of the center of mass from the previous link and not the general frame of reference, you have to multiply the vector with the transformation matrix (homogeneus) that you can obtain from the DH table

$$[r_{0,ci}, 1]^T =^0 A_1 \cdot^1 A_2 \ldots^{i-1} A_i [r_{i,ci}, 1]^T \tag{42}$$

we have that:

$$U = U_1 + U_2 + \cdots + U_n \tag{43}$$

We can find the gravity vector $g(q)$ as:

$$g(q) = \begin{bmatrix} \frac{\partial U}{\partial q_1} \\ \vdots \\ \frac{\partial U}{\partial q_n} \end{bmatrix} \tag{44}$$

Note: the gravity term is equal to zero ($g(q) = 0$) when:

- We are moving horizontally (constant U)
- We have static balancing (distribution of masses)
- We have mechanical compensation (system of springs or closed kinematic chains)

5.6.1 ex: Define suitable relations between the link masses, lengths, and CoM position given an expected gravity term

The steps are the following:

- Express the position of the CoMs wrt the origin frame (using the DH values to find the transformation matrices) ${}^0r_{ci} = {}^0A_i^i r_{ci}$
- Find $U_i = -m_i g r_{ci}$
- Write $g(q) = \frac{\delta U}{\delta q_i}$
- Impose $g(q) =$ expected gravity term and find the relations between the link masses, lengths and CoM positions

5.6.2 ex: Define a tight bound on the norm of the square matrix $\frac{\partial g}{\partial q}$

The steps are the following:

- First compute the gravity vector $g(q)$
- Then perform $\frac{\partial g}{\partial q} = \frac{{}^2\partial U}{\partial q^2}$
- Compute the norm as: $||\frac{\partial g}{\partial q}|| = \sqrt{\lambda_{max}}$ where λ_{max} is the maximum eigenvalue of the matrix $(\frac{\partial g}{\partial q}^T \frac{\partial g}{\partial q})$
- Find the bound α as the highest possible value of the norm (by setting arbitrarly q values)

5.7 Energy Conservation

The total robot energy is expressed as:

$$E = T + U = \frac{1}{2}\dot{q}^T M \dot{q} + U(q) \tag{45}$$

The evolution of the total energy over time is given by:

$$\dot{E} = \dot{q}^T u + \frac{1}{2}\dot{q}^T(\dot{M}(q) - 2S(q, \dot{q}))\dot{q} \tag{46}$$

If the potential energy is constant: $\dot{E} = \dot{q}^T u = \dot{L}$

5.8 Additional dynamic terms

5.8.1 Friction torque

The viscous friction torque is given by:

$$u_{V,i} = -F_{V,i}\dot{q}_i \tag{47}$$

The Coulomb friction torque is given by:

$$u_{C,i} = -F_{C,i}sign(\dot{q}_i) \tag{48}$$

5.8.2 Electrical motors

Motor i mounted on link $i-1$ (or before) often with its spinning axis aligned with joint axis i.
The mass is added to the link $i-1$ and the inertia is added to the robot kinetic energy.

5.8.3 Complete dynamic model

The complete dynamic model is given by:

$$(M(q) + B_m(q))\ddot{q} + C(q,\dot{q})\dot{q} + g(q) + F_v\dot{q} + F_c sign(\dot{q}) = \tau \tag{49}$$

where $B_m(q)$ is the motor inertia matrix, F_v is the viscous friction coefficient and F_c is the Coulomb friction coefficient and τ are motor torques after the reduction gears.
To introduce **elasticity** we can add a term $K(q)q$ to the left side of the equation.

5.9 Structural properties in dynamic models

The coriolis term $C(q,\dot{q})$ can be rewritten as $S(q,\dot{q})\dot{q}$ where S is the skew-symmetric matrix.
The matrix $\dot{M}(q) - 2S(q,\dot{q})$ is symmetric.

5.9.1 ex: Find matrix S that satisfies $\dot{M}(q) - 2S(q,\dot{q})$, checking if it's skew symmetric

The steps are the following:

1. Compute the coriolis matrix as in **??**
2. Compute the skew symmetric matrix as: $S(q,\dot{q}) = [S_1; ...; S_n]$ where S_i is the skew symmetric matrix obtained by multiplying $S_i = dq^T \cdot C_i$
3. Check if the matrix $\dot{M}(q) - 2S(q,\dot{q})$ is skew symmetric

5.9.2 ex: Find a different (non standard) Skew symmetric matrix S' that satisfies $\dot{M}(q) - 2S'(q, \dot{q})$ and a matrix S'' that does not satisfy the property

The steps are the following:

1. Compute the coriolis matrix as in **??**
2. Compute the skew symmetric matrix as: $S(q, \dot{q}) = [S_1; ...; S_n]$ where S_i is the skew symmetric matrix obtained by multiplying $S_i = dq^T \cdot C_i$
3. Find a different matrix $S' = S + SS$ that satisfies the property by adding a skew symmetric matrix SS to the original S (Note: $SS * \dot{q}$=0)
4. Find a matrix $S'' = S + SS$ that does not satisfy the property by adding to the original S a matrix that is not skew symmetric SS but still is a feasible factorization (i.e. $S''(q, \dot{q})\dot{q} = c(q, \dot{q})$, ie $SS * \dot{q} = 0$)

6 Linear parametrization of the dynamic model

Each link is characterized by 10 dynamic parameters:

- 1 mass m_i
- 3 coordinates of the center of mass $r_{c,i} = \begin{bmatrix} r^x_{c,i} & r^y_{c,i} & r^z_{c,i} \end{bmatrix}^T$
- 6 elements of the inertia matrix $I_{c,i} = \begin{bmatrix} I^{xx}_{c,i} & I^{xy}_{c,i} & I^{xz}_{c,i} & I^{yy}_{c,i} & I^{yz}_{c,i} & I^{zz}_{c,i} \end{bmatrix}^T$ (the upper triangular part)

Both Potential and Kinetic energy can be expressed as a linear combination of the dynamic parameters pi:

$$T_i = \frac{1}{2} m^i_i v^T_i \cdot^i v_i + \frac{1}{2} \omega^T_i I^i_{c,i} \omega_i + m^i_i r^T_{c,i} S({}^i v_i) *^i \omega_i \tag{50}$$

$$: U_i = -m_i g^T_0 r_i - g^T_0 \cdot^0 R_i m^i_i r_{c,i} \tag{51}$$

Since the dynamic model is linear on T and U, the dynamic model can be expressed as:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = Y_\pi(q, \dot{q}, \ddot{q})\pi = u \tag{52}$$

where $Y_\pi(q, \dot{q}, \ddot{q})$ is the regressor matrix that has a block upper triangular structure.
Important: The dynamic model can be rewritten as:

$$Y(q, \dot{q}, \ddot{q})a = Y_\pi(q, \dot{q}, \ddot{q})\pi \tag{53}$$

where a is the vector (p x 1) of the dynamic coefficients (ie the combination of the dynamic parameters that matter in the dynamic model)

6.1 Minimal parametrization

Determining the minimal parametrization means to find the smallest p that allows to express the dynamic model.

6.1.1 ex: Find the regressor matrix Y for a parametrized matrix M such that M=Ya

The steps are the following:

1. Obtain Y on MATLAB as `Y = jacobian(M, a)`
2. In mathematically terms we have to compute the derivative of M w.r.t. a

7 Newton-Euler method

7.1 Newton and Euler equations

7.1.1 Newton equation

The Newton equation is given by:

$$\sum f_i = m \cdot \dot{v}_c \tag{54}$$

where f_i are force acting on the link and v_c is the linear velocity of the center of mass. By the principle of action and reaction we have that:

$$f_i - f_{i+1} + m_i g = m_i \dot{v}_{c,i} \tag{55}$$

where f_i is the force from link $i-1$ to i, f_{i+1} is the force applied from i on $i+1$, m_i is the mass of the link and g is the gravity vector. **Note:** All vectors must be expressed in the same reference frame.

7.1.2 Euler equation

The Euler equation is given by:

$$\sum \mu_i = I \cdot \dot{\omega} + \omega \times I\omega \tag{56}$$

where μ_i are the torques acting on the link and ω is the angular velocity of the link and I is the inertia matrix.
By the principle of action and reaction we have that:

$$\tau_i - \tau_{i+1} + f_i \times r_{i-1,ci} + f_{i+1} \times r_{i,ci} = I_i \dot{\omega}_i + \omega_i \times (I_i \omega_i) \tag{57}$$

where τ_i is the torque acting on the link, τ_{i+1} is the torque acting on the next link, I_i is the inertia matrix of the link and ω_i is the angular velocity of the link.

7.2 Recursive Newton-Euler algorithm

The recursive Newton-Euler algorithm is used to compute the dynamic model of a robot. It works as follows:

7.2.1 Initialization

Set:

- ${}^0\omega_0 = 0$
- ${}^0\dot{\omega}_0 = 0$
- ${}^0a_0 = 0$
- $f_{N+1} = 0$
- $\tau_{N+1} = 0$

7.2.2 Forward recursion

- Compute ${}^i\omega_i$ and ${}^i\dot{\omega}_i$ as:

$$ {}^i\omega_i = {}^{i-1}R_i^T \cdot [{}^{i-1}\omega_{i-1} + \dot{q}_i^{i-1} z_{i-1}] \tag{58} $$

$$ {}^i\dot{\omega}_i = {}^{i-1}R_i^T \cdot [{}^{i-1}\dot{\omega}_{i-1} + \ddot{q}_i^{i-1} z_{i-1} + \dot{q}_i^{i-1}\omega_{i-1} \times {}^{i-1} z_{i-1}] \tag{59} $$

- Compute ${}^ia_i$ as:

$$ {}^ia_i = {}^{i-1}R_i^T \cdot ({}^{i-1}a_{i-1}) + {}^i\dot{\omega}_i \times {}^i r_{i-1,i} + {}^i\omega_i \times ({}^i\omega_i \times {}^i r_{i-1,i}) \tag{60} $$

- Compute ${}^ia_{ci}$ as:

$$ {}^ia_{ci} = {}^i a_i + {}^i\dot{\omega}_i \times {}^i r_{c,i} + {}^i\omega_i \times ({}^i\omega_i \times {}^i r_{c,i}) \tag{61} $$

7.2.3 Backward recursion

- Compute ${}^if_i$ as:

$$ {}^if_i = {}^i R_{i+1} \cdot {}^{i+1} f_{i+1} + m_i \cdot ({}^ia_{ci} - {}^i g) \tag{62} $$

- Compute ${}^i\tau_i$ as:

$$ {}^i\tau_i = {}^i R_{i+1} \cdot {}^{i+1}\tau_{i+1} + ({}^iR_{i+1}^{i+1} f_{i+1}) \times {}^i r_{i,ci} - {}^if_i \times ({}^ir_{i-1,i} + {}^ir_{c,i}) + I_i \cdot {}^i\dot{\omega}_i + {}^i\omega_i \times (I_i \cdot {}^i\omega_i) \tag{63} $$

7.2.4 Generalized forces

The generalized forces are given by:

$$ u_i = \begin{cases} {}^if_i \cdot {}^i * z_{i-1} & \text{if the joint is prismatic} \\ {}^i\tau_i^T \cdot {}^i * z_{i-1} & \text{if the joint is revolute} \end{cases} \tag{64} $$

7.3 Output examples

Using the MATLAB routine $Ne_\alpha(arg1, arg2, arg3)$ we can obtain the following outputs:

- **Complete inverse dynamics**: $Ne_{0_g}(q, dq, ddq) = M(q) \cdot ddq + C(q, dq) + g(q)$
- **Gravity term**: $Ne_{0_g}(q, 0, 0) = g(q)$
- **Coriolis term**: $Ne_0(q, dq, 0) = C(q, dq)$
- **I-th column of the inertia matrix**: $Ne_0(q, 0, e_i) = M_i(q)$ where e_i is the i-th column of the identity matrix
- **Generalized momentum**: $Ne_0(q, 0, \dot{q}) = M(q) \cdot \dot{q}$

Note: For the command $Ne_0(q, 0, 0)$, if a force F_e is applied to the end effector, the output is: $Ne_0(q, 0, 0) = J^T(q)F_e$ where J is the Jacobian matrix of the robot.

8 Basic control

There are two main types of control schemes:

- **Open loop control**: the control input is computed without any feedback from the system
- **Closed loop control**: the control input is computed using feedback from the system

Types of control systems base on uncertainty:

- **Feedback control**: small variations of parameters and initial conditions
- **Robust control**: large variations of parameters and initial conditions
- **Adaptive control**: improves performance online, adapting the control law to unknown range of uncertainties
- **Intelligent control**: control based on experience

The different types of control are based on the type of error and task can be observed in Figure **??**.

8.1 Control notation

We use a new notation for position and velocity that are now expressed as:

$$x = \begin{bmatrix} x_1 \\ \vdots \\ x_2 \end{bmatrix} = \begin{bmatrix} q_1 \\ \vdots \\ q_n \end{bmatrix} \tag{65}$$

control.png

Figure 1: Control system

8.2 Equilibrium

There are two types of equilibrium:

- **unforced equilibrium**: $u = 0$ zero velocity and no gravity forces on the robot
- **forced equilibrium**: $u = u(x)$ zero velocity and $u(x_e) = g(x_{e1})$

8.3 Stability of dynamical systems

Stability of x_e is defined as:

$$\forall \epsilon > 0, \exists \delta > 0 : ||x(t_0) - x_e|| < \delta \Rightarrow ||x(t) - x_e|| < \epsilon \tag{66}$$

8.3.1 Asymptotic stability

Asymptotic stability of x_e is defined as:

$$\exists \delta > 0 : ||x(t_0) - x_e|| < \delta \Rightarrow \lim_{t \to \infty} ||x(t) - x_e|| = 0 \tag{67}$$

Note: This definition is in the sense of Lyapunov

8.3.2 Exponential stability

Exponential stability of x_e is defined as:

$$\exists \delta, c, \lambda > 0 : ||x(t_0) - x_e|| < \delta \Rightarrow ||x(t) - x_e|| \leq ce^{-\lambda(t-t_0)}||x(t_0) - x_e|| \tag{68}$$

- Can estimate the time to approximately converge
- Typically this is a local property

Note: A necessary condition for x_e to be both globally asymptotically stable and exponentially stable is that it is the only equilibrium point of the system.

8.4 Lyapunov stability

8.4.1 Lyapunov candidate

A Lyapunov candidate is a function $V(x)$ that is positive definite that can be used to prove the stability of a system.
Examples of Lyapunov candidates are:

- For a dynamic system: $V(x) = T(x) + U(x)$ + eventual additional terms

8.4.2 ex: Write the Lyapunov candidate given a dynamic model and a control law

The steps are the following:

1. Given the dynamic models of the system we want to build the Lyapunov candidate V by adding all the terms in the system as follows:
   - For each mass m_i we have to add the kinetic energy $\frac{1}{2}m_i q_i^2$
   - For each spring k_i we have to add the potential energy $\frac{1}{2}k_i e_i^2$
   - For each virtual spring $K_{p,i}$ we have to add the potential energy $\frac{1}{2}K_{p,i}e_i^2$
   - For the gravity term we have to add the potential energy $-m_i g q_i$
   - Friction is not considered in the Lyapunov candidate
   - Dampers are not considered in the Lyapunov candidate
2. Then we build the differentiated version of V, $\dot{V}$. The terms can be differentiated as follows:
   - For the mass term
     - if m_i is in function of q, it becomes $\frac{1}{2}\dot{q}^T\dot{m}_i\dot{q} + \dot{q}^T m_i \ddot{q}$
     - else it becomes $\dot{q}^T m_i \ddot{q}$
   - For each spring, the term becomes $\dot{e}^T k_i e$
   - For each virtual spring, the term becomes $-\dot{q}^T K_{p,i} e$
   - For each gravitational term, the term becomes $-m_i g \dot{q}$
3. **Note**: the e in springs and virtul springs is usually different (in virtual spring is the differnece between desiredand actual value)
4. Prove that $\dot{V}(x) \leq 0$ along the trajectories of the system. To simplify the proof you can use the following property

$$\dot{q}^T(\dot{M}(q) - 2S(q,\dot{q}))\dot{q} = 0 \tag{69}$$

   We usually proceed in the simplifications by substituting $m_i\ddot{q}$ with the value from the control loop

8.4.3 Lyapunov stability

Lyapunov stability of x_e is defined as:
Sufficient condition of stability

$$\exists V \text{candidate} : \dot{V}(x) \leq 0 \text{along the trajectories of the system} \tag{70}$$

Sufficient condition of asymptotic stability

$$\exists V \text{candidate} : \dot{V}(x) < 0 \text{along the trajectories of the system} \tag{71}$$

Sufficient condition of instability

$$\exists V \text{candidate} : \dot{V}(x) > 0 \text{along the trajectories of the system} \tag{72}$$

8.5 LaSalle Theorem

La Salle's theorem states that if $\exists V$candidate such that: $\dot{V}(x) \leq 0$ along the trajectories of the system, then:
system trajectories are bounded and converge to the largest invariant set
$M \subseteq S = \{x \in \mathbb{R}^n | \dot{V}(x) = 0\}$
Corolloary: $M \equiv x_e \rightarrow$ asymptotic stability

8.6 Stability of dynamical systems

For general time-varying systems, the stability can be analyzed using the Lyapunov method.

8.6.1 Barbalat Lemma

Barbalat's lemma states that if:

- a function V(x,t) is lower bounded
- $\dot{V}(x,t) \leq 0$

then $\lim_{t\to\infty} \dot{V}(x,t) = 0$.
Corollary: if a Lyapunov candidate V(x,t) satisfies Barbalat's lemma, then the conclusion of LaSalle's theorem holds (ie. the system is asymptotically stable)

8.6.2 ex: Verify the unique asymptotic stability of a closed loop system in a given equilibrium

Ways to verify the asymptotic stability of a closed loop system:

- **Lyapunov + LaSalle**: We have to find a Lyapunov candidate V(x) **??** and prove that $\dot{V}(x) < 0$ and then apply LaSalle's theorem **??**
- **Linearization**: Analyzes the linearized version of the system dynamics, obtained by a first-order Taylor expansion around the desired closed-loop equilibrium point

Lyapunov + LaSalle :
The steps are the following:

1. Write (if not given) the dynamic model of the system
   - For system of masses: $\Rightarrow$ **??**
   - For a robot: $\Rightarrow$ **??**
2. Write the control law (if not given). (Note: if you have multiple dynamic equations there might be some conditions on the control law, depending on the relation between the dynamic equations)
3. Obtain the closed loop system equations (by equating the right side of the dynamic model(s) to the control law)

4. Find a Lyapunov candidate V(x) **??**. Note: one single Lyapunov candidate independently of the number of dynamic equations
5. Verify that $V(x)$ is positive for all states and for the desired state it is equal to zero (when $x = x_d$)
6. Compute $\dot{V}(x)$
7. Prove that $\dot{V}(x) < 0$
8. Prove that the equilibrium is the only one by applying LaSalle's theorem **??** (check that $\dot{V}(x) = 0$ only at the equilibrium)
9. Impose $\ddot{x} = 0$ and prove that this happens only for $x = x_d$

Linearization:
The steps are the following:

1. Write (if not given) the dynamic model of the system
   - For system of masses: $\Rightarrow$ **??**
   - For a robot: $\Rightarrow$ **??**
2. Bring to Laplace domain the dynamic model
3. Prove asymptotic stability of the linearized system with Routh criterion

8.6.3 ex: Find the unique equilibrium state for the closed-loop system under a given control

The steps are the following:

1. Write (if not given) the dynamic model of the system
   - for system of masses: $\Rightarrow$ **??**
   - for a robot: $\Rightarrow$ **??**
2. Write the control law (if not given). (Note: if you have multiple dynamic equations there might be some conditions on the control law, depending on the relation between the dynamic equations)
3. Obtain the closed loop system equations (by equating the right side of the dynamic model(s) to the control law)
4. Substitute in the closed loop system equations the values of the equlibrium state (the known one, hence $\dot{q} = 0$ and $\ddot{q} = 0$)
5. Solve the system of equations to find the equilibrium state q_d
6. Show global asymptotic stability from step 4 as in **??** (with Lyapunov + LaSalle steps)

9 Which control law?

If asked to design a control law without direct request of a specific method, you can use the following criteria to choose the correct one:

- To achieve **global exponential stabilization** of a state with **decoupled transient evolutions of the position error** → y feedback linearization control in the joint space
- To achieve **global asymptotic stabilization** of a state, **without knowledge of the robot inertia matrix** → a PD control with gravity cancellation or iterative gravity cancellation
- To achieve **exponential stabilization of the end-effector position** $p = p_d$ up to kinematic singularities → feedback linearization control in the Cartesian space
- **Global asymptotic stabilization** of a state for an **underactuated robot** → an approximate feedback linearization control

10 PD control

The **PD control** is a control scheme that uses the proportional and derivative terms of the error to compute the control input.
The goal is asymptotic stability of the closed-loop system:
$q = q_d$ and $\dot{q} = 0$
The **Control law** is given by:

$$u = K_p(q_d - q) + K_d(-\dot{q}) \tag{73}$$

where K_p and K_d are the proportional and derivative gains.

10.0.1 Th.1

In the absence of gravity the robot state $(q_d, 0)$ under the given PD joint control law is globally asymptotically stable.

10.1 PD control with gravity compensation

$g(q)$ in a robot is the gravity term, and is bounded as follows:

$$||\frac{\partial g}{\partial q}|| \leq \alpha \tag{74}$$

Note: gravity term is bounded only if:

- The robot has all revolute joints
- The robot has both types of joints but no prismatic variable in $g(q)$

- All prismatic joints have limited ranges

consequently:

$$||g(q) - g(q_d)|| \leq \alpha ||q - q_d|| \tag{75}$$

The **Control law** is given by:

$$u = K_p(q_d - q) - K_d\dot{q} + g(q_d) \tag{76}$$

Note: If a viscous friction $-F_v\dot{q}$ is present, the term $-K_d\dot{q}$ can be omitted.

10.1.1 Th.2

If $K_{P,m} > \alpha$ the state $(q_d, 0)$ under the given PD joint control law with gravity compensation is globally asymptotically stable.

10.2 Approximate gravity compensation

The approximate gravity compensation is given by:

$$u = K_p(q_d - q) - K_d(\dot{q}) + \hat{g}(q) \tag{77}$$

where $\hat{g}(q)$ is an approximation of the gravity term. It leeds to a closed loop equilibriuum whose uniqueness is not guaranteed.

10.2.1 ex: find the minimum values of the gains for a PD control with gravity compensation

The steps are the following:

1. Find the gravity term $g(q)$ for the robot
2. Find the bound α as in **??**
3. Impose $K_{P,m} > \alpha$
4. If we have viscous friction, we can impose $K_{D,m} = 0$

Note: The $Th.2$ just gives a sufficient condition, not a necessary one, we can observe the structure of the gravity term $g(q)$ and for each entry i, we can say that it is sufficient that $K_{P,i} > 0$ if the $i - th$ entry of the gravity term is:

- Constant
- Equal to zero

11 PID control

The **PID control** is a control scheme that uses the proportional, integral and derivative terms of the error to compute the control input (adds the integral term to the PD control to eliminate the steady-state error).
PID can be used to recover a position error due to absent or incomplete gravity compensation.
It is independent from the robot dynamics
The **Control law** is given by:

$$u = K_p(q_d - q) - K_d(\dot{q}) + K_i \int_0^t (q_d - q_\tau) d\tau \tag{78}$$

11.1 Saturated PID control

Global asymptotic stability is not guaranteed for PID control, but can be proven under lower bound assumption on the gains for a non-linear PID law:

$$u = K_p(q_d - q) - K_d(\dot{q}) + K_i \int_0^t \phi(q_d - q_\tau) d\tau \tag{79}$$

where ϕ is a saturation function.
Examples of saturation functions are:

- $\phi(x) = \begin{cases} \sin x & \text{if } |x| \leq \pi/2 \\ 1 & \text{if } x > \pi/2 \\ -1 & \text{if } x < -\pi/2 \end{cases}$
- $\phi(x) = \tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

12 Iterative Learning for gravity compensation

We can use the iterative method when:

- The robot has a complex (or unknown) dynamic model
- We don't need high position gain
- We don't want complex conditions on the gains

It can be used to derive sufficient conditions for global convergence with zero final error.
The **Control law** at the i-th iteration is given by:

$$u_i = \gamma K_p(q_d - q) - K_d(\dot{q}) + u_{i-1} \tag{80}$$

where γ is the learning gain, u_{i-1} is the feedforward term, $u_0 = 0$ is the easiest initialization of the feedforward term.
At steady state we have:

$$g(q_i) = \gamma K_p(q_d - q_i) + u_{i-1} = u_i \tag{81}$$

12.0.1 Th.3

If:

- $\lambda_{min}(K_p) > \alpha$
- $\gamma \geq 2$

then we are guaranteed that the sequence $q_0, .., q_i$ converges to q_d (and $\dot{q} = 0$) for any initial value (global convergence).
Note: The $Th.3$ just gives a sufficient condition, not a necessary one.

13 Trajectory tracking control

13.1 Inverse Dynamic control

Given a dynamic model of a robot $M(q)\ddot{q} + n(q, \dot{q}) = u$, (where n includes also friction and other disturbances) and a twice differentiable desired trajectory $q_d(t)$, we can obtain the desired control input u_d as:

$$u_d = M(q_d)\ddot{q}_d + n(q_d, \dot{q}_d) \tag{82}$$

If there are disturbances we need to use a feedback to make the control scheme more robust.
Possible implementations:

- **Offline**: Open loop
- **Online**: Closed loop

There are 2 steps in control design:

1. Compensation (feedforward) or cancellation (feedback) of nonlinearities
2. Synthesis of a linear control law stabilizing the trajectory error to zero

13.2 Trajectory controllers

Possible trajectory controllers are:

- Inverse dynamic compensation (FFW) + PD control

$$u = \hat{u}_d + K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}) \tag{83}$$

- Inverse dynamic compensation (FFW) + variable PD

$$u = \hat{u}_d + \hat{M}(q_d)[K_P(q_d - q) + K_D(\dot{q}_d - \dot{q})] \tag{84}$$

  or

- Feedback linearization + PD + FFW = **computed torque**

$$u = \hat{M}(q)[\ddot{q} + K_P(q_d - q) + K_D(\dot{q}_d - \dot{q})] + \hat{n}(q, \dot{q}) \tag{85}$$

- Feedback linearization + PID + FFW

$$u = \hat{M}(q)[\ddot{q} + K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}) + K_I \int_0^t (q_d - q_\tau) d\tau] + \hat{n}(q, \dot{q}) \tag{86}$$

Note: The global stabilization for the last two laws is obtained with $K_P > 0$ and $K_D > 0$

13.2.1 Feedback linearization

In nominal conditions ($\hat{M} = M$ and $\hat{n} = n$) we have:

$$u = M(q)\ddot{q} + n(q, \dot{q}) = M(q)a + n(q, \dot{q}) \tag{87}$$

So the global asymptotic stabilization of tracking error is:

$$a = \ddot{q}_d + K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}) \tag{88}$$

Under feedback linearization control, the robot has a dynamic behavior that is invariant, linear and decoupled in its whole state space ($\forall(q, \dot{q})$).
Linearity: error tends to 0 exponentially.
Decoupling: each joint coordinate evolves independently from the others.

13.2.2 Remarks

- Desired robot trajectory can be generated from Cartesian data $p_d(0), \dot{p}_d(0), \ddot{p}_d(0)$
- Real-time computation with Newton-Euler algorithm

13.3 Alternative global trajectory controller

$$u = M(q)\ddot{q} + S(q, \dot{q})\dot{q} + g(q) + F_V\dot{q}_d + K_P e + K_D \dot{e} \tag{89}$$

This formula above gives a global asymptotic stabilization of the tracking error, no cancellation of nonlinearities and no linear and decoupled behaviour.

13.4 Regulation special case

The regulation is a special case of trajectory tracking where q_d is constant.
The feedback linearization control law is given by:

$$u = M(q)[K_P(q_d - q) - K_D(\dot{q})] + c(q, \dot{q}) + g(q) \tag{90}$$

This is a solution to the regulation problem with exponential stability.
The alternative global trajectory controller is given by:

$$u = K_P(q_d - q) - K_D(\dot{q}) + g(q) \tag{91}$$

(It's a simple PD control with gravity compensation)

13.5 Repetitive motion tasks

The repetitive motion tasks are tasks that are executed repeatedly in a finite time.
The robot is reinitialized at the initial state at the beginning of each trial.
The control law is made of a non-model based part (often, a decentralized PD law) + a time-varying feedforward which is updated before every trial
The update of the feedforward term is done by:

$$v_{k+1}(s) = \alpha(s)u'_k(s) + \beta(s)v_k(s) \tag{92}$$

If a contraction condition can be enforced

$$||\beta(s) - \alpha(s)W(s)|| < 1 \tag{93}$$

then the control law is globally asymptotically stable.
Note: If the choice $\beta = 1$ satisfies the contraction condition, then we coverge with 0 tracking error. If $\alpha(s) = 1/W(s)$ then we converge in 1 iteration.

13.6 Application to robots

Recursive robot equation:

$$u = [B_m + M(q)]\ddot{q} + [F_v + S(q, \dot{q})]\dot{q} + g(q) \tag{94}$$

Control law (pre-learning):

$$u = K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}) + \hat{g}(q) \tag{95}$$

Learning filters:

$$W_i = \frac{q_i(s)}{q_{di}(s)} = \frac{K_{Pi} + K_{Di}s}{\hat{B}_{mi}s^2 + (\hat{F}_{vi} + K_{Di})s + K_{Pi}} \tag{96}$$

Initializaton of feedforward term (using best estimate):

$$v_1 = [\hat{B}_m + \hat{M}(q)]\ddot{q}_d + [\hat{F}_v + \hat{S}(q, \dot{q})]\dot{q}_d + \hat{g}(q) \tag{97}$$

13.6.1 Control with approximate feedback linearization

Dynamic model, its nominal part and (unstructured) uncertainty:

$$M(q)\ddot{q} + n(q, \dot{q}) = \tau \tag{98}$$

$$M = \hat{M} + \Delta M \tag{99}$$

$$n = \hat{n} + \Delta n \tag{100}$$

Model based feedback linearization:

$$\tau_{fl} = \hat{M}(q)a + \hat{n}(q, \dot{q}) \tag{101}$$

Resulting closed-loop dynamics with perturbation:

$$\ddot{q} = a + \delta(q, \dot{q}, a) \tag{102}$$

$$\delta = (M^{-1}\hat{M} - I)a + M^{-1}(\hat{n} - n) \tag{103}$$

Control law for tracking $q_d(t)$ (PD with feed forward + regressor ϵ_k):

$$a = u_k + \epsilon_k = K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}) + \ddot{q}_d + \epsilon_k \tag{104}$$

The regressor term is obtained with a Gaussian Process (GP) model:

$$\epsilon_k \circ \mathcal{N}(\mu(\hat{X}_k, \sigma^2(\hat{X}_k))) \tag{105}$$

13.6.2 ex: Achieve global exponential stabilization with decoupled transient evolution of positions errors e(q,t)

The steps are the following:

- Write the feedback linearization control law as:

$$u = M[-K_D\dot{q} + K_P(q_d - q)] + c(q, \dot{q}) + g(q) \tag{106}$$

- Obtain the desired error transient by choosing suitable gains K_D and K_P that satisfy the following equation:

$$\ddot{e}_i + K_{D,i}\dot{e}_i + K_{P,i}e_i = 0 \tag{107}$$

- Obtain $\dot{e}$ and $\ddot{e}$ from the given e equation by differentiation and substitute them in the previous formula.
- Solve by principle of polynomial identity for K_Ds and K_Ps

13.6.3 ex: Solve the inverse dynamics for a desired trajectory $q_d(t)$

The steps are the following:

1. Write the dynamic model of the robot
2. Substitute the desired trajectory in the dynamic model
3. Solve the equation for τ

Note: If you have more than one dynamic equation, you have to substitute the desired trajectory in all the equations, obtain the values of the other parameter (say θ) and its derivatives in terms of the desired trajectory and then substitute them in the equation for τ.

14 Adaptive control

We need adaptation in robot motion when we have:

- Poor knowledge of inertial payload
- Large uncertainties in robot dynamics parameters (m_i, I_i, r_{ci})

We assume that DH parameters are known (including link lengths).
Main methodologies:

- **Linear parametrization** of robot dynamics
- **Nonlinear control law** of dynamic type (nonlinear control based on feedback linearization)

14.1 Adaptive Control Law

First adaptive method: on-line modification with reference velocity.
$\dot{q}_r = \dot{q}_d + \Lambda(q_d - q)$ where typically $\Lambda = K_D^{-1} K_P$.
Adaptive control law:

$$u = M(q)\ddot{q}_r + S(q, \dot{q})\dot{q} + g(q) + F_V \dot{q}_r + K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}) \tag{108}$$

with adaptation law:

$$\dot{\hat{a}} = \Gamma Y^T(q, \dot{q}, \dot{q}_r, \ddot{q}_r)(\dot{q}_r - \dot{q}) \tag{109}$$

14.2 Th.4

The introduced adaptive controller makes the tracking error along the desired trajectory globally asymptotically stable.

14.2.1 Properties

If the desired trajectory $q_d(t)$ is persistently exciting, then the estimates of the dynamic parameters converge to the true values.
Condition of persistent excitation:

- For linear systems: The number of frequency components in the desired trajectory should be at least twice as large as the number of unknown coefficients
- For nonlinear systems: The condition can be checked only a posteriori (a squared motion integral should always be positive bounded from below)

14.3 Adaptive regulation

Adaptation in case q_d is constant:

- The adaptive control law remains the same:

$$u = \hat{M}(q)\ddot{q}_r + \hat{S}(q, \dot{q})\dot{q}_r + \hat{g}(q) + \hat{F}_V \dot{q}_r + K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}) \tag{110}$$

  and adaptation law:

$$\dot{\hat{a}} = \Gamma Y^T(q, \dot{q}, \dot{q}_r, \ddot{q})(\dot{q}_r - \dot{q}) \tag{111}$$

- If we have an adaptive version of the trajectory tracking controller instead it would only need the adaptation on the gravity term.

$$u = \hat{M}(q)\ddot{q}_d + \hat{S}(q,\dot{q})\dot{q}_d + \hat{g}(q) + \hat{F}_V\dot{q}_d + K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}) \tag{112}$$

and adaptation law:

$$g(q) = G(q)a_g \tag{113}$$

$$u = G(q)\hat{a}_g + K_P(q_d - q) + K_D(\dot{q}) \tag{114}$$

$$\dot{\hat{a}}_g = \gamma G^T(q)(\frac{2e}{1 + 2||e||^2} - \beta\dot{q}) \tag{115}$$

where $e = q_d - q$ and γ and β are positive constants

14.3.1 ex: How many states has the adaptive dynamic controller?

The number of states is the number of coefficients in a. Note that if you have also a separated gravity term, you have to consider also the coefficients of the gravity term (number of states $= |a_m| + |a_g|$)

14.3.2 ex: Find the adaptive control law that obtains global asymptotic tracking of a trajectory $q_d(t)$

The steps are the following:

1. Write the dynamic model of the robot and obtain the linear parametrization Y
2. Write the control law and adaptation law as in **??** (removing eventually not-necessary terms)
3. Write the formula for q_r as $\dot{q}_r = \dot{q}_d + \Lambda(q_d - q)$ where $\Lambda = K_D^{-1}K_P$

14.3.3 ex: Find the reduced states adaptive control law that obtains global asymptotic tracking of a trajectory $q_d(t)$ assuming dynamic coefficients a_m are known

The steps are the following:

1. Write the gravity term as $g(q) = G(q)a_g$
2. Write the control law and adaptation law as in **??**
3. Write the formula for q_r as $\dot{q}_r = \dot{q}_d + \Lambda(q_d - q)$ where $\Lambda = K_D^{-1}K_P$

15 Cartesian control

We use the Cartesian control when the goal is the asymptotic stabilization of the end-effector position:
$p = p_d, \dot{q} = \dot{q}_d = 0 \leftrightarrow \dot{p} = \dot{p}_d = 0$
Note:
if $m = n$, then $\dot{q} \implies \dot{p} = 0$
if $m < n$ then the goal is not uniquely associated to a complete robot state.
The **Cartesian regulation control law** is given by:

$$u = J^T(q)K_P(p_d - p) - K_D\dot{q} + g(q) \tag{116}$$

15.1 Th.5

Under the control law in **??**, the robot state will converge asymptotically to the set $A = \{\dot{q} = 0, g : K_p(p_d - f(q)) \in N(J^T(q))\}$

15.2 Corollary

For a given initial state $(q(0), \dot{q}(0))$, if the robot does not encounter any singularity of $J^T(q)$ during its motion, then there is asymptotic stabilization to one single state or to a set of states such that $e_p = 0, \dot{q} = 0$

15.3 Cartesian control variant

Control law (PD control plus gravity cancellation in joint space):

$$u = J^T(q)[K_P(p_d - p) - K_D\dot{p}] + g(q) \tag{117}$$

15.4 Feedback linearization in Cartesian space

Algorithm: differentiate the output(s) as many times as needed up to the appearance of (at least one of) the input torque(s) u , then verify if it is possible to solve for the input.

- $y = f(q)$
- $\dot{y} = J(q)\dot{q}$
- $\ddot{y} = J(q)\ddot{q} + \dot{J}(q)\dot{q} = J(q)M(q)^{-1}(u - c(q, \dot{q}) - g(q)) + \dot{J}(q)\dot{q}$

where $\ddot{q}$ is taken from the dynamic model

15.4.1 Th.6

Theorem for a non-redundant robot, it is possible to exactly linearize and decouple the dynamic behavior at the Cartesian level if and only if $\det(J(q)) \neq 0$

15.4.2 Linearization in right coordinates

From the control law,

$$u = M(q)J^{-1}(q)a + c(q, \dot{q}) + g(q) - M(q)J^{-}1(q)\dot{J}(q)\dot{q} \tag{118}$$

we can derive:

$$\ddot{y} = \ddot{p} = J(q)M^{-1}(u - c(q, \dot{q}) - g(q)) + \dot{J}(q)\dot{q} = a \tag{119}$$

From the closed-loop equation in joint space we obtain:

$$\ddot{q} = J^{-1}(q)a - J^{-1}(q)\dot{J}(q)\dot{q} \tag{120}$$

Note: the design of a Cartesian trajectory tracking control is completed by setting:

$$a_i = \ddot{p}_{di} + K_{Di}(\dot{p}_{di} - \dot{p}_i) + K_{Pi}(p_{di} - p_i) \tag{121}$$

Note: For p_d constant, the control law is:

$$u = M(q)J^{-1}(q)[K_P e_P - K_D J(q)\dot{q}] + c(q, \dot{q}) + g(q) - M(q)J^{-}1(q)\dot{J}(q)\dot{q} \tag{122}$$

15.4.3 ex: Find a robot configuration q and consequent p (that is not p_d) such that the robot will not move when at rest in q

The steps are the following:

1. Write the Cartesian control law as in **??**
2. Find the singularities of the Jacobian
3. Substitute the values of the singularities in the forward kinematics to find the corresponding end effector position
4. Substitute the value of q, $\dot{q} = 0$ in the control law and obtain its reduced form
5. Equate the reduced form to the model dynamics (where you substitute q and $\dot{q}$ and prove that $\ddot{p} = 0$

16 Robot interaction with the environment

16.1 Which type of control law?

If asked to design a control law without direct request of a specific method, you can use the following criteria:

- If you have knowledge of the specific geometry of the environment and the robot is in contact with it, you can use a hybrid force/velocity control
- If you have no knowledge of the specific geometry of the environment and the robot is in contact with it, you can use impedance control

16.2 Robot compliance

Robot compliance can be:

- **Passive**: the robot is compliant by structure
- **Active**: the robot is able to control the compliance with
  - Admittance control
  - Impedance control
  - Stiffness/compliance control
  - Hybrid control

in all cases for physical interaction tasks, the desired motion specification and execution should be integrated with complementary data for the desired force.

16.3 Ideally constrained contact

If geometry is known hybrid force/velocity control is the best choice. In the ideal case, with a infinitely stiff environment, we can assume $f_e = -f_r$ where f_e is the force exerted by the environment and f_r is the force exerted by the robot (normally w.r.t environment).

16.4 Constrained robot dynamics

Suppose that the task variables are subject to $m < n$ (bilateral) geometric constraints in the general $k(r) = 0$ and define

$$h(q) = k(f(q)) = 0 \tag{123}$$

the constrained robot dynamics can be derived using again the Lagrange formalism, by defining an augmented Lagrangian as:

$$L(q, \dot{q}, \lambda) = T(q, \dot{q}) - U(q) - \lambda^T h(q) \tag{124}$$

where the Lagrange multipliers λ (a m-dimensional vector) can be interpreted as the generalized forces that arise at the contact when attempting to violate the constraints.
By applying the Euler-Lagrange equations we obtain the constrained robot dynamics as:

$$M(q)\ddot{q} + c(q, \dot{q}) + g(q) = u + A^T(q)\lambda \tag{125}$$

where A(q) is the Jacobian of the constraints $h(q)$, in fact: $A(q) = \frac{\partial h}{\partial q}$
We can obtain the value of λ by doing the following:

- Differentiate the constraint $h(q) = 0$ with respect to time (twice), obtain $\ddot{h} = A(q)\ddot{q} + \dot{A}(q)\dot{q} = 0$
- Substitute the value of $\ddot{q}$ from the constrained robot dynamics

- Obtain the value of λ by solving the equation

$$\lambda = (AM^{-1}A^T)^{-1}(AM^{-1}(c+g-u) - \dot{A}\dot{q}) \tag{126}$$

The final constrained dynamic model can be rewritten as:

$$M(q)\ddot{q} = [I - A^T(q)(A_M^{\#}(q))^T](u - c(q,\dot{q}) - g) - M(q)A_M^{\#}(q)\dot{A}(q)\dot{q} \tag{127}$$

where $A_M^{\#}(q)$ is the Moore-Penrose pseudo-inverse of $A(q)$:

$$A_M^{\#}(q) = M(q)^{-1}A^T(q)(A(q)M^{-1}(q)A^T(q))^{-1}$$

16.5 Reduced robot dynamics

By imposing m constraints $h(q) = 0$ on the n generalized coordinates q, it is also possible to reduce the description of the constrained robot dynamics to a $n - m$ dimensional configuration space.
Along with that, we must remember that when dealing with constraint dynamics, the general form of $h(q)$ is:

$$h(q) = \begin{bmatrix} q_1(t) - k_1 \\ q_2(t) - k_2 \\ q_3(t) - k_3 \\ \vdots \\ q_n(t) - k_n \end{bmatrix} \tag{128}$$

but, when solving the exercises, you consider only the constrained joints.
For example if you have a constraint on the first joint, your $h(q)$ will be:

$$h(q) = \begin{bmatrix} q_1(t) - k_1 \\ q_2(t) - 0 \\ q_3(t) - 0 \\ \vdots \\ q_n(t) - 0 \end{bmatrix} = \begin{bmatrix} 0 \\ q_2(t) \\ q_3(t) \\ \vdots \\ q_n(t) \end{bmatrix}$$

which automatically translates in simply considering $h(q) = q_1(t) - k = 0$, as the constraints for $q_2, q_3, ..., q_n$ are not present.
Start from a constraint matrix A(q) and select D(q) such that

$$\begin{bmatrix} A(q) \\ D(q) \end{bmatrix} \tag{129}$$

in a non-singular matrix, then the matrices $E(q)$ and $F(q)$ can be found in this way:

$$\begin{bmatrix} A(q) \\ D(q) \end{bmatrix}^{-1} = \begin{bmatrix} E(q) & F(q) \end{bmatrix} \tag{130}$$

Then we define the vector of pseudovelocities v as:

$$v = D(q)\dot{q} \tag{131}$$

From that we can obtain $\dot{q}$ and $\ddot{q}$ as:

$$\dot{q} = F(q)v \tag{132}$$

$$\ddot{q} = F(q)\dot{v} - (E(q)\dot{A}(q) + F(q)\dot{D}(q))F(q)v \tag{133}$$

From that we can rewrite the constrained robot dynamics as:

$$(F^T MF)\dot{v} = F^T(u - c(q, \dot{q}) - g + M(E\dot{A} + F\dot{D})Fv) \tag{134}$$

and the lagrangian multipliers becomes:

$$\lambda = E^T(MF\dot{v} - M(E\dot{A} + F\dot{D})Fv + c + g - u) \tag{135}$$

N.B. In this case, your usual $M(q)$ becomes $F^T(q)M(q)F(q)$ which is the *reduced inertia matrix.*

17 Impedance control

The impedance control imposes a desired dynamic behavior to the interaction between robot end-effector and environment specified through a complete set of mass-spring-damper equations.
The dynamic model of a robot in contact is given by:

$$M(q)\ddot{q} + c(q, \dot{q})\dot{q} + g(q) = u + J_r^T(q)F_r \tag{136}$$

where $J_r(q)$ is the analyitic jacobian (from forward kinematics) and $F_r = T_r^{-T}(\psi)F$.
Note: In case of constraint forces instead of contact forces we can rewrite the equation as:

$$M(q)\ddot{q} + c(q, \dot{q})\dot{q} + g(q) = u + A^T(q)\lambda \tag{137}$$

where $A(q)$ is the Jacobian of the constraints and λ is the reaction force intenity due to the constraints.

17.1 Dynamic model in Cartesian coordinates

The dynamic model in Cartesian coordinates is given by:

$$M_r(q)\ddot{r} + S_r(q, \dot{q})\dot{r} + g_r(q) = J_r^{-T}(q)u + F_r \tag{138}$$

where:

- $M_r(q) = J_r^{-T}(q)M(q)J_r^{-1}(q)$
- $S_r(q, \dot{q}) = J_r^{-T}(q)S(q, \dot{q})J_r^{-1}(q) - M_r(q)\dot{J}_r(q)J_r^{-1}(q)$
- $g_r(q) = J_r^{-T}(q)g(q)$

17.2 Design of the control law

The design happens in two steps:

1. Feedback linearization in the Cartesian space (with force measure)

$$u = J_r^T(q)[M_r(q)a + S_r(q,\dot{q})\dot{r} + g_r(q) - F_r] \tag{139}$$

2. Imposition of a dynamic impedance model

$$M_m(\ddot{r} - \ddot{r}_d) + D_m(\dot{r} - \dot{r}_d) + K_m(r - r_d) = F_r \tag{140}$$

   where M_m, D_m, K_m are the desired mass, damping and stiffness.
   that is realized by choosing:

$$a = \ddot{r}_d + M_m^{-1}[D_m(\dot{r}_d - \dot{r}) + K_m(r_d - r) + F_r] \tag{141}$$

17.2.1 Control law in joint coordinates

$$\begin{aligned} u =& M(q)J_r^{-1}(q)\ddot{r}_d - \dot{J}_r(q)\dot{q} + M_m^{-1}[D_m(\dot{r}_d - r) + K_m(r_d - r)] + \\ & S(q,\dot{q})\dot{q} + g(q) + J_r^T(q)[M_r(q)M_m^{-1} - I]F_r \end{aligned} \tag{142}$$

Note: We can choose the parameter to have a desired dynamic:

- Large M_m and small K_m when we want low contact forces
- Small M_m and large K_m when we want good tracking of desired motion trajectory
- Damping coefficients D_m are used then to shape transient behaviors

17.2.2 Simplifications

If we assume $M_m = M_r$ the control law becomes:

$$u = M(q)J_r^{-1}(q)\ddot{r}_d - \dot{J}_r(q)\dot{q} + S(q,\dot{q})\dot{q} + g(q) + J_r^T(q)[D_m(\dot{r}_d - \dot{r}) + K_m(r_d - r)] \tag{143}$$

This is a pure motion control law applied also during interaction, but designed so as to keep limited contact forces at the end-effector leave.
Note: If the impedance model (now, non-linear) is still supposed to represent a real mechanical system, then in correspondence to a desired non-constant inertia there should be Coriolis and centrifugal terms such that the control law becomes:

$$u = M(q)J_r^{-1}(q)\ddot{r}_d - \dot{J}_r(q)J_r^{-1}(q)\dot{r}_d + S(q,\dot{q})J_r^{-1}(q)\dot{r}_d + g(q) + J_r^T(q)[D_m(\dot{r}_d - \dot{r}) + K_m(r_d - r)] \tag{144}$$

which guarantees convergence to zero tracking error wher $F_r = 0$.
We have a further simplification if r_d is constant. In this case, the control law becomes:

$$u = g(q) + J_r^T(q)[-D_m\dot{r}_d + K_m(r_d - r)] \tag{145}$$

17.3 General case $F_r \neq 0$

For analysis we model the environment as an elastic element:

$$F_r = K_e(r_e - r) \tag{146}$$

where $\dot{r} = \ddot{r} = 0$ a closed loop system equilibrium is given by:

$$K_m(r_d - r) + K_e(r_e - r) \tag{147}$$

which has unique solution

$$r = (K_m + K_e)^{-1} K_m r_d + K_e r_e) = r_E \tag{148}$$

17.3.1 Equivalence with RCC device

Theorem: If $(r_d - r) \simeq J_r(q)(q_d - q)$ and $g(q) = 0$ and $D_m = 0$
then the control law is equivalent to the one of a RCC device:

$$u = J_r^T(q) K_m J_r(q)(q_d - q) = K(q)(q_d - q) \tag{149}$$

17.3.2 ex: Check that the tip acceleration has the same direction of any force F in the plane applied to the tip of the robot

The steps are the following:

1. Write the dynamic model in Cartesian coordinates as in **??**
2. Isolate the acceleration $\ddot{r}$ from the dynamic model
3. If the acceleration is solely dependent on the force F, then the direction of the acceleration is the same as the direction of F

17.3.3 ex: Write the relation between p an q given the model dynamics in joint space

- Start from the relation $\dot{p} = J(q)\dot{q}$
- By duality obtain $\tau_q = J^T(q)\tau_p$
- Substitute the value of τ_q in the dynamic model, and τ_p as $(\tau_1 \dots \tau_n)^T$ and obtain the value of $J^T(q)$
- Use it to find p

17.3.4 ex: Design an impedance control law for a robot

The steps are the following:

1. Write the dynamic model in Cartesian coordinates
2. Write the impedance model
3. Check if the appartent cartesian matrix M_m (the one of the impedance model) is equal to the natural inertia matrix M_r (this could be specified in the text or be a consequence of the absence of force/torque sensors)
4. If so, substitute f_r obtained from the impedance model inside the dynamic model
5. From the dynamic model, isolate and obtain u (obtaining the control law)

17.3.5 ex: Find the equilibrium point for impedance control

The steps are the following:

- Write the impedance control law at steady-state ($\dot{r} = \ddot{r} = 0$)
- Isolate and find r (equilibrium point)
- Write the control law as in **??** and substitute the values of steady state ($\dot{r} = \ddot{r} = 0$)
- Compute u with the r previously obtained: u_E

Note: Remember that when there is an external *constant* force F_c and you are asked to find u_E or τ_E, along with x_E, you simply find them by:

$$x_E = x_d + K^{-1} F_c \tag{150}$$

$$u_E = J^T(q_E) F_c + g(q_E) \tag{151}$$

18 Hybrid control

The hybrid control law is designed in ideal conditions, but now unconstrained directions of motion and constrained force directions are defined in a more direct way using a task frame formalism.
All non-ideal conditions (compliant surfaces, friction at the contact, errors in contact surface orientation) are handled explicitly in the control scheme by a geometric filtering of the measured quantities.

18.1 Natural and artificial constraints

18.1.1 Natural constraints

In ideal conditions we can define inside the task space:

- **End-effector motion** (v, ω) is prohibited in $6 - k$ directions
- **Reaction forces/torques** (f, μ) are absent in k directions

18.1.2 Artificial constraints

The **artificial constraints** can be specified by:

- **End-effector velocities** (v, ω) around k directions where motion can occur
- **Contact forces/torques** (f, μ) $6 - k$ directions where admissible environment reactions can occur

18.2 Hybrid force/velocity control

The general parametrization of hybrid task can be described by:

1. Robot-environment contact type:

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = T(s)\dot{s} \tag{152}$$

$$\begin{bmatrix} f \\ \mu \end{bmatrix} = Y(s)\lambda \tag{153}$$

2. Robot dynamics:

$$M(q)\ddot{q} + S(q, \dot{q}) + g(q) = u + J^T(q)[f, \mu]^T \tag{154}$$

3. Robot kinematics:

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = J(q)\dot{q} \tag{155}$$

The **control objective** is to impose a desired task evolution to the parameters s of motion and λ of force.
The control law is designed in two steps:

1. Extract linearization and decoupling in the task frame by feedback

$$u = (MJ^{-1}T, -J^TY)(a_s, a_\lambda)^T + MJ^{-1}(\dot{T}\dot{s} - \dot{J}\dot{q}) + S\dot{q} + g \tag{156}$$

2. (Linear) design of a_s and a_λ so as to impose the desired dynamic behavior to the errors $e_s = s_d - s$ and $e_\lambda = \lambda_d - \lambda$

$$a_s = \ddot{s}_d + K_D(\dot{s}_d - \dot{s}) + K_P(s_d - s) \tag{157}$$

$$a_\lambda = \lambda_d + K_I \int (\lambda_d - \lambda)dt \tag{158}$$

   where values for s, $\dot{s}$ are extracted from measures of q and $\dot{q}$ equating the descriptions of the end-effector pose and velocity "from the robot side" (direct and differential kinematics) $\dot{s} = T^{\#}J(q)\dot{q}$ and "from the environment side" while λ is obtained from force/torque measures at end effector $\lambda = Y^{\#}(f, m)^T$

Note: We assume $n = m$ and $J(q)$ out of singularity

18.3 Force control via impedance model

In a force-controlled direction of the hybrid task space, when the contact stiffness is limited, one may use impedance model ideas to explicitly control the contact force.
The impedance model is chosen as:

$$m_m\ddot{x} + d_m\dot{x} + k_s(x - x_d) = f_d \tag{159}$$

After a feedback linearization the command a_x is designed as:

$$a_x = (1/m_m)[(f_d - f_s) - d_m\dot{x}] \tag{160}$$

18.3.1 Sources of inconsistency

The main sources of inconsistency in force and velocity measurements can be:

- Presence of friction at the contact
- Compiance in the robust structure and/or at the contacy
- Uncertainty on environment geometry at the contact

18.4 Estimation of unknown surface

- Normal = nominal direction of measured force
- Tangent = nominal direction of measured velocity
- Tangent direction and friction angle can also be extimated with recursive least square method

18.4.1 ex: Design the natural and artificial constraints for a hybrid control law. How many control loops? How many degres of freedom are needed?

The steps are the following:

1. Design the task frame (usually, align the z-axis with the end effector)
2. Start by designing the natural constraints:
   - Define the velocities (linear and angular) that are prohibited in the task space $(6 - k)$
   - Define the forces/torques that are absent in the task space (k)
3. Design the artificial constraints:
   - Define the velocities (linear and angular) that are allowed in the task space (k)
   - Define the forces/torques that are present in the task space $(6 - k)$
4. The number of control loops is equal to the number of artificial constraints (6, but specifically k control loops on motion components and $6 - k$ on force components)
5. The number of degrees of freedom needed is one per control loop

18.4.2 ex: Design a hybrid control law for a robot

The steps are the following:

1. Write the control law as in **??**: $M(q)\ddot{q} + S(q,\dot{q})\dot{q} + g(q) = u + J^T(q)u_f$ where u_f is the vector of forces/torques expressed in the original frame (same as the model)
2. Compute the rotation matrix R from origin frame to contact plane frame
3. Rewrite the dynamic model in the contact plane frame by multiplying the elements of the model by R, such that: $R^T M(q)R\ddot{q} + R^T S(q,\dot{q})R\dot{q} + R^T g(q) = R^T u + R^T J^T(q)u_f$ where $R^T u_f$ is the vector of forces/torques expressed in the contact plane frame
4. Write the control loop for the force control

19 Visual servoing

Visual servoing refers to the use information acquired by vision sensors (cameras) for feedback control of the pose/motion of a robot.
The first phase is the **image processing**. The **vision system** provides set-point references to a cartesian motion controller.
Possible visual servoing schemes:

- **Image-based visual servoing**: The control law is designed directly in the image space - 2D plane. The steps are:
  - Image acquisition
  - Feature extraction
  - Comparison with desired values (error)
  - Generation of motion of camera/robot
- **Position-based visual servoing**: The control law is designed in the robot task space - in 3D pose (cartesian control law)

19.1 Camera Model

The camera model is given by:

- Set of lenses (thin lenses, pinhole, catadioptic lens)
- Matrix of light-sensitive elements
- Frame grabber

lens1.png

Figure 2: thin lens camera model

19.1.1 Thin lens camera model

Fundamental equation of a thin lens:

$$\frac{1}{Z} + \frac{1}{z} = \frac{1}{\lambda} \tag{161}$$

where Z is the distance between the object and the lens, z is the distance between the image plane and the lens and λ is the focal length of the lens.

19.1.2 Pinhole camera model

The pinhole camera model is given by:

$$u = \lambda \frac{X}{Z} \tag{162}$$

$$v = \lambda \frac{Y}{Z} \tag{163}$$

with $P = (X, Y, Z)$ the point in the 3D space and (u, v, λ) the point in the image plane.

19.1.3 Interaction matrix

Given a set of features parameters f =$[f_1, f_2, ..., f_n]$, we look for the (kinematic) differential relation between motion imposed to the camera and motion of features on the image plane

$$\dot{f} = J(\cdot)[V, \Omega]^T \tag{164}$$

where (V,Ω) is a six dimensional linear/angular velocity vector and $J(\cdot)$ is the **interaction matrix**.
To compute the interaction matrix:

lens2.png

Figure 3: pinhole camera model

1. From the perspective equation we obtain:

$$\begin{bmatrix} \dot{u} \\ \dot{v} \end{bmatrix} = \begin{bmatrix} \frac{\lambda}{z} & 0 & -\frac{u}{Z} \\ 0 & \frac{\lambda}{Z} & -\frac{v}{Z} \end{bmatrix} \begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix} = J_1(u, v, \lambda) \begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix} \tag{165}$$

2. From kinematic relation, instead, we can obtain:

$$\begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 & 0 & -Z & Y \\ 0 & -1 & 0 & Z & 0 & -X \\ 0 & 0 & -1 & -Y & X & 0 \end{bmatrix} \begin{bmatrix} V \\ \Omega \end{bmatrix} = J_2(X, Y, Z) \begin{bmatrix} V \\ \Omega \end{bmatrix} \tag{166}$$

3. Thus we obtain the interaction matrix J_p as

$$\begin{bmatrix} \dot{u} \\ \dot{v} \end{bmatrix} = J_1 J_2 \begin{bmatrix} V \\ \Omega \end{bmatrix} = \begin{bmatrix} -\frac{\lambda}{Z} & 0 & \frac{u}{Z} & \frac{uv}{\lambda} & -(\lambda + \frac{u^2}{\lambda}) & v \\ 0 & -\frac{\lambda}{Z} & \frac{v}{Z} & \lambda + \frac{v^2}{\lambda} & -\frac{uv}{\lambda} & -u \end{bmatrix} \begin{bmatrix} V \\ \Omega \end{bmatrix} = J_p(u, v, Z) \begin{bmatrix} V \\ \Omega \end{bmatrix} \tag{167}$$

Note: When requested to obtain the interaction matrix coming from multiple points we have variables u_i, v_i and Z_i for each point i, λ is the same for all points.

19.1.4 Other interaction matrices

Other examples of interaction matrices are:

- Distance between two points in the image plane

$$\begin{aligned} d &= \frac{1}{d}\begin{bmatrix} u_1 - u_2 & v_1 - v_2 & u_2 - u_1 & v_2 - v_1 \end{bmatrix} \begin{bmatrix} \dot{u}_1 \\ \dot{v}_1 \\ \dot{u}_2 \\ \dot{v}_2 \end{bmatrix} \\ &= J_P(u_1, u_2, v_1, v_2) \begin{bmatrix} J_{p1}(u_1, v_1, Z_1) \\ J_{p2}(u_2, v_2, Z_2) \end{bmatrix} \begin{bmatrix} V \\ \Omega \end{bmatrix} \end{aligned} \tag{168}$$

- Image moments:

$$m_{ij} = \int\int x^i y^j dxdy \tag{169}$$

19.2 Robot Differential Kinematics

In the **eye-in-hand** case, the motion to be imposed to the camera coincides with the desired end-effector linear/angular velocity.

$$\begin{bmatrix} V \\ \Omega \end{bmatrix} = J_m(q)\dot{q} = J_m(q)u \tag{170}$$

where $J_m(q)$ is the geometric jacobian or the NMM jacobian of a manipulator.
We obtain:

$$\dot{f} = J_p(f, Z)J_m(q)u = J(f, Z, q)u \tag{171}$$

where $J(f, Z, q)$ is the **Image jacobian**.
In general we can obtain the control (for $IBVS$) as:

$$u = J^{\#}(\dot{f}_d + ke) + (I - J^{\#}J)u_0 \tag{172}$$

and this will exponentially stabilize the error to 0 (up to singularities).

19.2.1 Control With Task Sequencing

Approach: regulate only one (some) feature at the time, while keeping "fixed" the others by unobservable motions in the image plane.
The steps are the following:

1. Select the feature to be regulated
2. Compute the control law for the selected feature

$$u = J_1^{\#}k_1e_1 + (I - J_1^{\#}J_1)u_0 \tag{173}$$

3. Compute the control law for the other features

$$u = (I - J_1^{\#}J_1)J_2^T k_2 e_2 \tag{174}$$

20 Actuation Faults

There are mainly 6 problems in the diagnosis of faults:

- **Fault Detection:** the fault is detected
- **Fault Isolation:** discriminate which particular fault f has occurred out of a (large) class of potential ones
- **Fault Identification:** the fault is quantified
- **Fault Accomodation:** the control law is modified to take into account the fault
- **FDI (Fault Detection and Isolation):** the fault is detected and isolated simultaneusly
- **Fault Tolerant Control:** the control law is modified to take into account the fault (passively or actively)

20.1 Fault Types

- Instantaneus
- Incipient
- Intermittent
- Concurrent

The dynamic in case of fault is given by:

$$M(q)\ddot{q} + c(q,\dot{q}) + g(q) + F_V\dot{q} + F_S sgn(\dot{q}) = u - u_f \tag{175}$$

where u_f is the fault vector, F_V is the viscous friction, F_S is the static friction.
Possible fault expressions:

- Total fault: $u_{f,i} = u_i$
- Partial fault: $u_{f,i} = \epsilon u_i$
- Saturation fault: $u_{f,i} = u_i - sgn(u_i)u_{max}$
- Bias fault: $u_{f,i} = b_i$

Note: We assume that the dynamic of the robot is known.

20.2 Generalized Momentum

The generalized momentum is defined as:

$$p = M(q)\dot{q} \tag{176}$$

and its dynamic is given by:

$$\dot{p} = u - \alpha(q,\dot{q}) - u_f \tag{177}$$

where

$$\alpha_i(q,\dot{q}) = -\frac{1}{2}\dot{q}^T\frac{\partial M}{\partial q_i}\dot{q} + g_i(q) + F_{V,i}\dot{q}_i + F_{S,i}sgn(\dot{q}_i) \tag{178}$$

20.3 FDI solution

Definition of the residual vector r as:

$$r = K[\int (u - \alpha(q, \dot{q}))dt - p] \tag{179}$$

where:

- K is a gain matrix
- $\alpha(q, \dot{q}) = -\frac{1}{2}\dot{q}^T \frac{\partial M}{\partial q}\dot{q} + g(q) + F_V\dot{q} + F_S sgn(\dot{q})$
- p is the generalized momentum $p = M(q)\dot{q}$

with perfect model knowledge the dynamics of r is:

$$\dot{r} = -Kr + Ku_f \tag{180}$$

20.3.1 Residual generator

The residual generator is a system that generates the residual vector r. We have:

$$\dot{\hat{p}} = u - \alpha(q, \dot{q}) + K(p - \hat{(}p)) \tag{181}$$

$$r = K(\hat{p} - p) \tag{182}$$

and $e_{obs} = u_f - r$

20.3.2 ex: Design a residual function r_i for a fault u_i

The steps are the following:

1. Compute the dynamic of the robot (or system), ex: $M(q)\ddot{q} + c(q, \dot{q})\dot{q} + g(q) = u - u_f$ where each entry u_i of the vector u_f is the fault of the i-th actuator (if present, 0 otherwise)
2. Write the residual function as:
$$r_i = K[\int (u_i - \alpha_i(q, \dot{q}) - r_i)dt - p_i] \tag{183}$$
3. Compute $alpha_i(q, \dot{q})$ as in **??**
4. Compute p_i as:
$$p = M(q)\dot{q} \tag{184}$$
5. Substitute the values in the residual function

20.3.3 ex: Give the expressions of the terms appearing in the definition of the residual vector r that allows detection and isolation of collisions.

The steps are the following:

1. Write the equation of the a possible contact force F_K acting on a link as $r \simeq \tau_K = J_K^T F_K$
2. Analyze the situation for each link:
   - Compute the forward kinematics up to the end link k
   - Compute the jacobian J_K of the end link k
   - Find the forces that cannot be detected as the null space of the jacobian J_K
   - Find the forces that can be detected but with wrong isolation as the forces that produce at least one zero in the residual vector

20.3.4 ex: Detect a collision through the energy-based and momentum-based methods

Energy-based method The energy-based method, in general, fails to detect a collision when the colliding force F_c is orthogonal to the velocity v_c of the contact point.

This means computing the following formula:

$$v_c^T F_c = (J_c(q)\dot{q})^T F_c \tag{185}$$

If the formula above is equal to zero, then no collision is detected.

Note that the subscript c denotes the collision point, meaning that if the exercise asks for the detection of a collision at a specific point, then the Jacobian J_c should be computed for that point.

If, for example, you have n contact points, then you'd have to compute:

$$\begin{aligned} v_{c1}^T F_{c1} &= (J_{c1}(q)\dot{q})^T F_{c1} \\ v_{c2}^T F_{c2} &= (J_{c2}(q)\dot{q})^T F_{c2} \\ &\dots \\ v_{cn}^T F_{cn} &= (J_{cn}(q)\dot{q})^T F_{cn} \end{aligned}$$

Momentum-based method In the momentum-based method, the collision is detected when F_c is not in the nullspace of the transpose of the Jacobian J_c, yielding the following formula:

$$\tau_c = J_c^T(q) F_c = 0 \tag{186}$$

In other words, if you're able to find the contact force F_c in the nullspace of the transpose of the Jacobian J_c, then the momentum-based method will detect a collision and also correctly isolate it.

A simple example:

$$1)\ \mathcal{N}\left\{\boldsymbol{J}_{c1}^T\right\} = \mathcal{N}\left\{\begin{pmatrix} -1 & 1 \\ 0 & 1 \end{pmatrix}\right\} = \emptyset \Rightarrow \boldsymbol{F}_{c1} = \begin{pmatrix} -1 \\ 0 \end{pmatrix} \notin \mathcal{N}\left\{\boldsymbol{J}_{c1}^T\right\}$$

hence:

$$\boldsymbol{\tau}_{c1} = \boldsymbol{J}_{c1}^T \boldsymbol{F}_{c1} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \text{ [Nm]}$$

$$2)\ \mathcal{N}\left\{\boldsymbol{J}_{c2}^T\right\} = \mathcal{N}\left\{\begin{pmatrix} -1 & 0 \\ 0 & 0 \end{pmatrix}\right\} = \left\{\begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \end{pmatrix}\right\} \Rightarrow \boldsymbol{F}_{c2} = \begin{pmatrix} -1 \\ 0 \end{pmatrix} \in \mathcal{N}\left\{\boldsymbol{J}_{c2}^T\right\}$$

hence:

$$\boldsymbol{\tau}_{c2} = \boldsymbol{J}_{c2}^T \boldsymbol{F}_{c2} = 0$$

In this case, only the second contact point (2) is in a collision state, as the torque τ_{c2} is equal to zero.

21 Extra exercises

21.0.1 ex: Uniform time scaling

If the motion is unfeasible and we need to apply uniform time scaling, the steps are the following:

1. Compute $K_i = \sqrt{\frac{\tau_i - g_i}{\tau_{max_i} - g_i}}$
2. Compute the new scaled time $t_i' = K_i t_i$
3. Compute the new scaled torques $\tau_i(t_i') = \frac{\tau_i(t_i) - g_i(q(t_i))}{K_i^2} + g_i(q(t_i))$

21.0.2 ex: Find q value that causes algorithmic singularity

The steps are the following:

1. Compute the Jacobian J for each of the tasks
2. Build the extended Jacobian as: $J_e = \begin{bmatrix} J_1 \\ J_2 \end{bmatrix}$
3. Compute the determinant of the extended Jacobian
4. Find the q value that causes the determinant to be zero
5. Substitute the q value in the Jacobians
6. Compute the rank of both the original jacobians and the extended one
7. If the rank of the extended jacobian i.e. $rank(J_e) < rank(J_1) + rank(J_2)$ is less than the sum of the ranks of the original jacobians, we have an algorithmic singularity.

21.0.3 ex: Check if two tasks are in conflict

The steps are the following:

1. Compute the Jacobian J for each of the tasks
2. Build the extended Jacobian as: $J_e = \begin{bmatrix} J_1 \\ J_2 \end{bmatrix}$
3. Build the extended desired velocity as: $\dot{r}_e = \begin{bmatrix} \dot{r}_1 \\ \dot{r}_2 \end{bmatrix}$
4. If the extended velcity is not in the range of the extended jacobian, the tasks are in conflict

21.0.4 ex: Find the minimum time of execution of a trajectory for a single link

The steps are the following:

1. Write the expression of the dynamic model
2. If we can identify the tkind of motion (e.g. bang-to-bang, bang-coast-bang, cubic polynomial smooth traj.) then
   (a) Write equations related to the motion the robot needs to perform (e.g. bang-to-bang, bang-coast-bang, cubic polynomial smooth traj.)
   (b) Find acceleration $\ddot{q}$ based on robot motion equation computed at previous step
   (c) Substitute it in the dynamic model $\rightarrow$ this will be the new τ
   (d) The new tau will be composed of two terms : a linear contribution u_a due to acceleration and a sinusoidal contribution u_g due to gravity.
   (e) Compute minimum time T based only on the acceleration contribution u_a by inverse formulas (reason explained next line)
   (f) **Note**: The faster will be the assigned trajectory (i.e, the smaller the total motion time T), the more will the acceleration term grow and dominate the gravitational term, which does not change being dependent only on the configuration q
3. else if the dynamic model is parametrized by a single acceleration and the motion to be executed has an unknown profile (e.g, not a bang-to-bang profile)
   (a) define two constants a and b s.t

$$-U_1 <= a * \ddot{q} <= U_1 \tag{187}$$

   and

$$-U_2 <= b * \ddot{q} <= U_2 \tag{188}$$

   (b) find $\ddot{q}$ as a function of the inequalities defined in previous step

(c) Express $\ddot{q}$ bound as

$$\max(-U_1/a, -U_2/b) = \ddot{q}^- \leq \ddot{q} \leq \ddot{q}^+ = \min(U_1/a, U_2/b) \tag{189}$$

(d) Check that the expressed conditions about $\ddot{q}^+$ and $\ddot{q}^-$ are able to perform any desired motion transfer (e.g. rest-to-rest motion)

(e) In case a rest-to-rest motion needs to be enforced, impose equations from moto uniformemente acceleration and find values of T and Ts

(f) For rest-to-rest motion end to be enforced, we set $\ddot{q}^- * Ts + \ddot{q}^+ * (T - Ts) = 0$

(g) For net displacement to be achieved, $1/2 * \ddot{q}^- * (Ts)^2 - 1/2 * \ddot{q}^+ * (T - Ts)^2 = displacement$

(h) Find the values of T and Ts

21.0.5 ex: Find the dynamic model of a system of masses spring and dampers

There are two ways to solve this problem:

- **Lagrangian method**: Write down the kinetic and potential energy of the system and write the dynamic model from there
- **Newton method**: Balance the forces and write the dynamic model

Newton method:
The steps are the following:

1. Write an equation for each body of the system
2. Include in the equation the following forces:
3.
   - the force of the spring $k_i(x_i - x_{i-1})$
   - the force of the damper $d_i(\dot{x}_i - \dot{x}_{i-1})$
   - the force of the gravity $m_i g$
   - the force of the acceleration $m_i \ddot{x}_i$
   - the force of the external input u_i
   - the force of the friction $f_i \dot{x}_i$
   - the force of the contact c_i
4. Obtain something like:

$$m_i \ddot{x}_i + k_i(x_i - x_{i-1}) + d_i(\dot{x}_i - \dot{x}_{i-1}) + m_i g + f_i \dot{x}_i + c_i = u_i \tag{190}$$

21.0.6 ex: Find the torque τ that realizes given objectives

The steps are the following:

1. Write the dynamic model of the robot
2. Isolate the torque τ from the dynamic model
3. If asked to minimize the squared norm of joint accelerations $H = 1/2||\ddot{q}||^2$:
   - find the value of $\ddot{q}$ to substitute in the dynamic model as:
   $$\ddot{q} = J(q)^{\#}(\ddot{r}_d + \dot{J}(q)\dot{q}) + (I - J(q)^{\#}J(q))\ddot{q}_0 \tag{191}$$
4. If asked to minimize the squared norm of absolute joint accelerations (joint accelerations but in absolute coordinates) $H = 1/2||\ddot{q_a}||^2$:
   - Find the transformation matrix T such that $\ddot{q_a} = T\ddot{q}$
   - Common example in a RRR robot is:
   $$T = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \tag{192}$$
   - Compute the weighting matrix W for the pseudoinverse as $W = T^T T$
   - Find the value of $\ddot{q_a}$ to substitute in the dynamic model using the weighted pseudoinverse:
   $$\ddot{q} = J(q)^{\#}_W(\ddot{r}_d + \dot{J}(q)_w\dot{q}) + (I - J(q)^{\#}_w J(q)_w)\ddot{q}_0 \tag{193}$$
5. If asked to minimize the squared norm of the inertia weighted joint accelerations $H = 1/2\ddot{q}^T M(q)\ddot{q}$:
   - Use the inertia weighted pseudoinverse
   - Find the value of $\ddot{q}$ to substitute in the dynamic model using the inertia weighted pseudoinverse:
   $$\ddot{q} = J(q)^{\#}_M(\ddot{r}_d + \dot{J}(q)_M\dot{q}) + (I - J(q)^{\#}_M J(q)_M)\ddot{q}_0 \tag{194}$$

22 APPENDIX

22.1 Pseudo-inverse properties

The properties of the pseudo-inverse are:

- $JJ^{\#}J = J$
- $J^{\#}JJ^{\#} = J^{\#}$
- $(JJ^{\#})^T = JJ^{\#}$
- $(J^{\#}J)^T = J^{\#}J$

22.2 Weighted pseudo-inverse properties

The properties of the weighted pseudo-inverse are:

- $JJ^{\#}J = J$
- $J^{\#}JJ^{\#} = J^{\#}$
- $(JJ^{\#})^T = JJ^{\#}$

22.3 DH frames

22.3.1 Assign axis

- z_i along the direction of joint i+1
- x_i along the common normal between z_{i-1} and z_i
- y_i completing the right-handed frame

22.3.2 DH table

- α_i is the angle from z_{i-1} to z_i about x_i
- a_i is the distance from z_{i-1} to z_i along x_i
- d_i is the distance from x_{i-1} to x_i along z_{i-1}
- θ_i is the angle from x_{i-1} to x_i about z_{i-1}

22.4 Trajectories

22.4.1 Cubic Polynomial

The cubic polynomial is defined as:

$$q(\tau) = a\tau^3 + b\tau^2 + c\tau + d \tag{195}$$

We further define the **doubly normalized polyinomial** as:

$$q_N(\tau) = q_0 + \Delta(q)(a\tau^3 + b\tau^2 + c\tau + d) \tag{196}$$

Special case if both initial and final velocity are zero (rest-to-rest) we have:

$$q(\tau) = q_0 + \Delta(q)(-2\tau^3 + 3\tau^2) \tag{197}$$

22.4.2 Quintic Polynomial

The quintic polynomial is defined as:

$$q(\tau) = a\tau^5 + b\tau^4 + c\tau^3 + d\tau^2 + e\tau + f \tag{198}$$

We further define the **doubly normalized polyinomial** as:

$$q_N(\tau) = q_0 + \Delta(q)(a\tau^5 + b\tau^4 + c\tau^3 + d\tau^2 + e\tau + f) \tag{199}$$

Special case if $v_{in} = v_{fin} = 0$ and $a_{in} = a_{fin} = 0$ (rest-to-rest) we have:

$$q(\tau) = q0 + \Delta(q)(6\tau^5 - 15\tau^4 + 10\tau^3) \tag{200}$$

22.5 Non-smooth trajectories

22.5.1 Bang-Bang Trajectory

The bang-bang trajectory is defined as:

$$q(t) = \begin{cases} \frac{a_{max}t^2}{2} & t \in [0, t_s) \\ -\frac{a_{max}(T-t)^2}{2} + v_{max}T - \frac{v_{max}^2}{a_{max}} & t \in [T - t_s, T] \end{cases} \tag{201}$$

$$\dot{q}(t) = \begin{cases} a_{max}t & t \in [0, t_s) \\ -a_{max}(T - t_s) + v_{max} & t \in [T - t_s, T] \end{cases} \tag{202}$$

$$\ddot{q}(t) = \begin{cases} a_{max} & t \in [0, t_s) \\ -a_{max} & t \in [T - t_s, T] \end{cases} \tag{203}$$

22.5.2 ex: Finding minimum time

To find the minimum time given max velocity and max acceleration we need to find the max of all the possible time computation for the bang-bang trajectory.

1. Compute the total distance or length (L or $\Delta(q)$) fro each joint
2. Compute the possible values of time (for each joint) as: $T_{min} = \sqrt{\frac{4L}{a_{max}}}$
3. Note: We can do that if we have initial and final velocity equal to zero
4. (we obtain that from $v^2 = v_0^2 + 2a\Delta(q)$)
5. Find the maximum value of the possible values of time

22.5.3 Bang-Coast-Bang Trajectory

The bang-coast-bang trajectory is defined as:

$$q(t) = \begin{cases} \frac{a_{max}t^2}{2} & t \in [0, t_s) \\ v_{max}t - \frac{v_{max}^2}{2a_{max}} & t \in [t_s, T - t_s) \\ -\frac{a_{max}(T-t_s)^2}{2} + v_{max}T - \frac{v_{max}^2}{a_{max}} & t \in [T - t_s, T] \end{cases} \tag{204}$$

$$\dot{q}(t) = \begin{cases} a_{max}t & t \in [0, t_s) \\ v_{max} & t \in [t_s, T - t_s) \\ -a_{max}(T - t_s) + v_{max} & t \in [T - t_s, T] \end{cases} \tag{205}$$

$$\ddot{q}(t) = \begin{cases} a_{max} & t \in [0, t_s) \\ 0 & t \in [t_s, T - t_s) \\ -a_{max} & t \in [T - t_s, T] \end{cases} \tag{206}$$

22.5.4 ex: Finding t_s and t_c

To find the values of t_s and t_c we need to impose the following constraints:

$$t_s = \frac{v_{max}}{a_{max}} \tag{207}$$

$$t_c = \frac{La_{max} + v_{max}^2}{a_{max}v_{max}} - 2 * t_s \tag{208}$$

The final time is given by $t_f = t_c + 2t_s$

22.5.5 ex: Finding minimum time

To find the minimum time given max velocity and max acceleration we need to find the max of all the possible time computation for the bang-coast-bang trajectory.

1. Compute the total distance or length (L or $\Delta(q)$) fro each joint
2. Compute the possible values of time (for each joint) as:
   - $T_{min} = \frac{La_{max}+v_{max}^2}{a_{max}v_{max}}$
   - $T_{min} = \frac{L}{v_{max}}$
3. Find the maximum value of the possible values of time

22.6 Energy

22.6.1 Kinetic energy

The kinetic energy of a spring is given by:

$$T = \frac{1}{2}(x_1 - x_2)^T K(x_1 - x_2) \tag{209}$$

The kinetic energy of a body accelerated by an acceleration $\ddot{q}$ is given by:

$$T = \frac{1}{2}\dot{q}^T M\dot{q} \tag{210}$$

Note: when adding those terms in the Lyapunov function we need to consider the eventual control virtual springs Kp and Kd.

22.6.2 Potential energy

The potential energy is given by gravity.

22.7 Laplace stability

- The system is stable if all the poles of the transfer function have negative real part i.e. the poles lie in the left half plane

22.7.1 Transfer function

The transfer function is defined as:

$$G(s) = \frac{Y(s)}{U(s)} \tag{211}$$

where $Y(s)$ is the output and $U(s)$ is the input

22.7.2 Poles

The poles of the transfer function are the values of s that make the denominator of the transfer function equal to zero

22.8 Routh Hurwitz criterion

The Routh Hurwitz criterion is a necessary and sufficient condition for the stability of a system.
The criterion is based on the coefficients of the characteristic polynomial of the system.
The criterion states that a system is stable if and only if all the coefficients of the characteristic polynomial are positive.
The criterion is based on the construction of a table called the Routh table.
The Routh table has $n + 1$ (where n is the maximum power) rows and is constructed as follows:

- The first row of the table is the coefficients of the characteristic polynomial (of the even powers)
- The second row is the coefficients of the characteristic polynomial (of the odd powers)
- The other rows are computed as follows:
  - The first element of the row is computed as:

$$a_{i,1} = -\frac{det \begin{bmatrix} a_{i-2,1} & a_{i-2,2} \\ a_{i-1,1} & a_{i-1,2} \end{bmatrix}}{a_{i-1,1}} \tag{212}$$

  - The other elements of the row are computed as:

$$a_{i,j} = -\frac{det \begin{bmatrix} a_{i-2,j-1} & a_{i-2,j} \\ a_{i-1,j-1} & a_{i-1,j} \end{bmatrix}}{a_{i-1,1}} \tag{213}$$

The system is stable if and only if all the elements of the first column of the Routh table are positive.

23 GLOSSARY

- **Self-motion**: Motion of a robot that does not change the position of the end-effector

- **Nominal condition**: The nominal condition is the condition in which the robot is supposed to be in.